HW#3 데이터통신 프로그래밍 과제

Fault-Tolerant 협업 Worker Ring — 반복되는 Crash · Restart · Rejoin 속에서도 정확히 끝나는 분산 작업

(Thread Pool · Mutex / Lock Ordering · Condition Variable · Heartbeat 장애 감지 · Checkpoint 복제 · Takeover · Epoch(Fencing) · Rejoin · Exactly-once)

○ **DUE**	2026.10.26(월) 23:59 (Assigned: 2026.10.13(화) 10:30)
○ **How**	1개 파일 제출 – G조이름HW3.zip (조별 한 명만 제출) ex) G2HW3.zip 모든 소스파일 및 결과파일 전체를 압축한 G조이름HW3.zip으로 제출

0. 개요 및 핵심 개념

HW#1은 여러 노드가 일을 나누는 "분산" 구조, HW#2는 한 서버가 공유 자원을 안전하게 나눠 쓰는 "동시성" 구조였다. HW#3은 여기에 **"장애 내성(Fault Tolerance)"**을 더한다. 일을 하던 노드가 갑자기 죽었다가 다시 살아나도 전체 작업이 **빠짐없이, 중복 없이** 끝나야 한다.

구체적으로, **물리적 원격 호스트(AWS, GCP 등)의 Coordinator 1개**와 **로컬 PC의 Worker 10개**가 1부터 1억(10^8)까지의 소수 개수를 **Task 1,000개**로 나누어 계산한다. Worker 10개는 시작할 때 고리(Ring)로 연결되고, 계산 진행 상황(Checkpoint)을 링의 다음 Worker에게 백업한다. 같은 로컬 PC의 **Chaos Launcher**가 무작위로 고른 Worker를 반복해서 crash시키고 얼마 뒤 다시 살린다(**임의의 순간 최대 2개**). Coordinator는 Heartbeat로 죽은 Worker를 찾아 그 일을 다른 Worker에게 넘기고(Takeover), 돌아온 Worker를 재합류(Rejoin)시킨다. 최종 결과는 **1,000 Task 모두 정확히 한 번 반영, 합계 5,761,455**이어야 한다.

crash는 **가상(virtual)**으로 구현한다. 연결(TCP)은 끊지 않고, crash 통보를 받은 Worker가 스스로 내부 상태를 모두 지우고 아무것도 보내지 않는다(침묵). Coordinator는 Launcher와 통신하지 않으며 **Heartbeat가 끊긴 것만 보고** 판단한다.

0-1. 과제 한눈에 보기 (Outline)

이 과제에서 무엇을 하나요?

- 원격 호스트의 **Coordinator** 1개와 로컬의 **Worker 10개**로 이루어진 분산 시스템을 만든다. Worker들은 시작할 때 서로 고리(**Ring**) 모양으로 연결된다.
- 1부터 1억까지의 소수 개수를 세는 큰 계산을 **Task 1,000개**로 나누어, Coordinator가 Worker들에게 나눠 주고 결과를 모은다. Task 하나는 **연속된 수 100,000개**를 맡는다(1억 ÷ 1,000). 예: Task 0 = [1, 100,000], Task 3 = [300,001, 400,000], Task 999 = [99,900,001, 100,000,000].
- Worker는 받은 Task를 다시 **step 10개**로 잘라 차례로 계산한다. step 하나는 **연속된 수 10,000개**다(100,000 ÷ 10). 예: Task 5의 step 7 = [560,001, 570,000]. (1-1)
- 계산 도중 **Chaos Launcher**가 Worker를 **무작위로 골라 crash**시키고, 얼마 뒤 다시 살린다. 같은 Worker가 여러 번 crash 할 수 있으며, **임의의 순간 최대 2개**의 Worker까지 동시에 crash 상태일 수 있다.
- crash한 Worker는 하던 일을 모두 잊는다. Coordinator는 이를 감지해 그 일을 다른 Worker에게 넘기고(**Takeover**), 돌아온 Worker는 **재합류(Rejoin)**해 새 일을 받는다.
- 목표는 어떤 crash가 일어나도 **모든 Task가 정확히 한 번씩** 반영되어 정답 **5,761,455**가 나오는 것이다.
- 개념은 실제 장애(노드가 죽었다 살아남)로 설명하지만, **구현은 가상(virtual)으로** 한다: 연결은 끊지 않고 Worker가 내부 상태를 지우고 침묵한다.
- HW#2에서 배운 Thread Pool, Mutex, Lock Ordering, Condition Variable을 그대로 사용한다.

이 문서가 HW#3의 **공식 명세**다. 예제의 시각·Task 번호·측정값은 설명을 위한 예시이며 실제 실행에서는 달라진다.

> **"Worker 10개가 링을 이루어 1억까지의 소수를 나누어 세는 동안, 무작위로 고른 Worker들이 몇 번이고 죽었다 살아나도(동시에 최대 2개) 최종 답 5,761,455가 정확히 한 번씩만 더해져 나오게 만든다."**

구분	내용
만들 프로그램 (3종)	**Coordinator** 1개(원격 호스트) · **Worker** 프로그램(로컬에서 10개 프로세스로 실행) · **Chaos Launcher** 1개(로컬, 장애 주입기)
Coordinator	Worker 10개를 등록하고 Ring을 구성시킨 뒤, 1억까지의 수를 **Task 1,000개**로 나눠 분배·합산한다. **Heartbeat**로 죽은 Worker를 찾아 그 일을 **링의 다음 살아 있는 Worker**에게 넘기고(Takeover), 돌아온 Worker를 **재합류(Rejoin)**시킨다.
Worker	받은 Task(수 100,000개)를 step 단위(수 10,000개씩, step 1~10)로 계산하고, step마다 진행 상황(**Checkpoint**)을 링의 다음 Worker에게 **직접** 보낸다. crash 통보를 받으면 기억을 모두 잃고 침묵하며, restart 통보를 받으면 재합류해 새 Task를 받는다.
Chaos Launcher	Worker 10개를 실행하고, **무작위로 고른 Worker**에게 **무작위 시점**에 CRASH, 무작위 시간 뒤 RESTART를 통보한다. 동시에 crash 상태인 Worker는 최대 2개로 유지한다.
성공 기준	Chaos Launcher가 무작위로 crash를 주입하는 실행에서도 **1,000 Task 모두 DONE · 모든 Task가 정확히 1번 반영 · 합계 5,761,455**, 그리고 crash마다 감지 → 인계 → 재합류 → 새 Task 수행이 로그로 확인될 것

0-2. 핵심 개념

개념	설명
Heartbeat · Failure Detector	Worker는 RUNNING일 때 1초마다 `HB i` 를 보낸다. Coordinator의 Failure Detector(FD)는 1초마다 lastHB를 검사해 **3초 이상 없으면 SUSPECT, 5초 이상 없으면 DEAD**로 판정한다(감지 지연 5~6초).
Checkpoint (CKPT) · Buddy	Worker i는 Task의 step을 끝낼 때마다 누적값을 **고정 Buddy(i) = i mod 10 + 1**에게 `CKPT k e s partial` 로 보낸다. Buddy는 Task별 최신 1개만 보관한다.
Takeover (다음 ALIVE)	DEAD가 된 Worker d가 owner인 **모든** Task를 링 순서 d+1, d+2, …에서 처음 만나는 **ALIVE** Worker에게 넘긴다(DEAD·SUSPECT는 건너뜀). 받은 Worker는 CKPT가 있으면 그다음 step부터, 없으면 step 1부터 계산한다.
Epoch · Commit	Task의 담당자가 바뀔 때마다 epoch를 +1 한다(ASSIGN = 1, TAKEOVER마다 +1). Coordinator는 RESULT의 epoch와 보낸 Worker가 현재 epoch·owner와 같을 때만 **Commit**(DONE 확정, 합계 반영)하고, 아니면 `REJECT_STALE` 로 거부한다 → **이중 Commit 0건**.
Rejoin	RESTART 통보를 받은 Worker는 `REJOIN i` 를 보내고, Coordinator가 `WELCOME` 하면 이전 기억 없이 새 Task를 요청한다. 같은 Worker가 몇 번이든 crash → rejoin을 반복할 수 있다.
Virtual crash · Chaos Launcher	Launcher가 `CRASH` 를 통보하면 Worker는 로컬 Queue·진행 중 Task·Checkpoint Store를 지우고 침묵한다(연결 유지, EOF 없음). 10~15초 뒤 `RESTART` 를 통보한다. victim은 무작위, 동시에 crash 상태는 최대 2개.
Thread Pool · Lock · CV	HW#2와 동일: Coordinator는 Dispatcher 6개 고정 Thread Pool + Request Queue, Task별 Mutex, Lock Ordering(Worker Table Lock → Task Lock), 모든 대기는 Condition Variable.

표기: Worker 번호 i, j(Buddy), d(죽은 Worker), b(인계 대상)는 1~10 (링 순서에서 10 다음은 1) · Task 번호 $0 \le k \le 999$ · Task 안의 step 번호 $1 \le s \le 10$ (CKPT는 $1 \le s \le 9$, RESULT는 s=10 이후) · epoch $e \ge 1$ (ASSIGN = 1, TAKEOVER마다 +1) · Dispatcher Dn ($1 \le n \le 6$). 시각은 특별한 말이 없으면 START 기준 초(sec)의 예시 값이다. Chaos Launcher의 t0(READY 10개 수신 시각)는 START 직후라 차이가 보통 0.1초 이내이며 예시에서는 같다고 본다. 예시의 Task 번호(152, 160, 405 등)는 설명용으로 고른 값이라 실제 오름차순 배정 순서(초당 약 10개)와 정확히 맞지 않을 수 있다. Launcher 로그와 Coordinator 로그의 대조는 두 로그에 모두 찍힌 실제 시각(HH:MM:SS.mmm)으로 한다.

0-3. 시스템 구조도 및 배포 (필수)

> **※ 배포 구조 (반드시 준수)**
>
> - Coordinator는 반드시 별도의 **물리적 원격 호스트(AWS, GCP 등)**에서 실행한다(보안 그룹에서 포트 허용, 실험 후 인스턴스 중지/삭제).
> - **Worker 10개와 Chaos Launcher는 같은 로컬 PC 한 대**에서 실행한다. Launcher가 Worker 10개를 각각 독립 프로세스로 직접 실행하며, Worker i의 Peer 포트는 **7000 + i**이다.

- IP · 포트 · Worker 번호 · seed는 실행 인자 또는 설정 파일로 지정한다(하드코딩 금지). 정식 실행(제출 로그)은 `--step-delay 100` (ms)으로, 원격 구성에서 수행한다. 개발은 로컬 한 대에서 해도 된다.
- 로그 시각은 실제 시각(HH:MM:SS.mmm)으로 기록하고, 시간대(UTC 또는 KST)를 통일해 Readme.txt에 적는다.

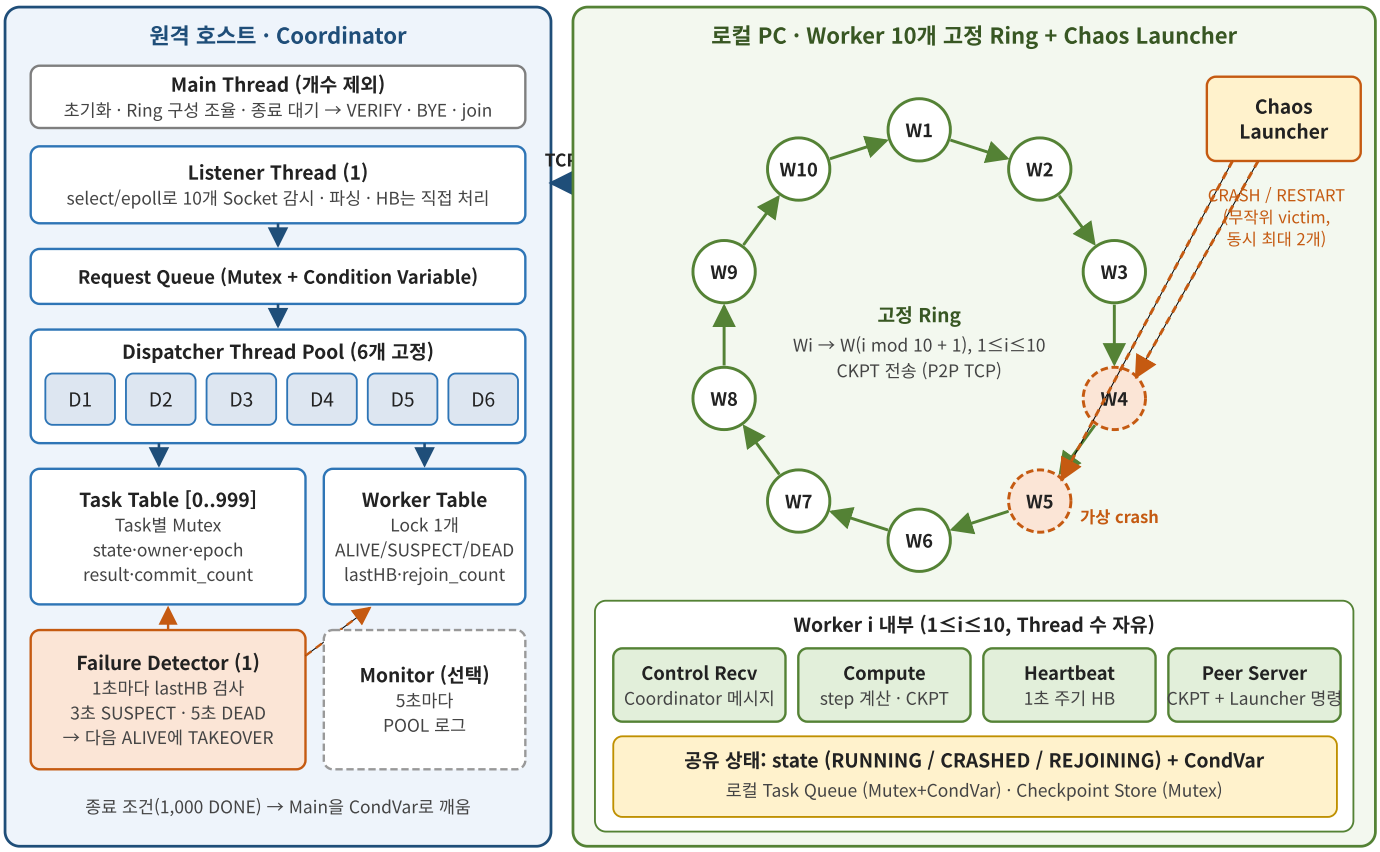


그림 1. 시스템 구조 — 원격 Coordinator(왼쪽), 로컬 Worker 10개의 고정 Ring과 Chaos Launcher(오른쪽). 예: W4 · W5가 동시에 가상 crash

연결 (모두 TCP, 1≤i≤10)	개수	용도
Coordinator ↔ Worker i	10	모든 제어 메시지 (HELLO, RING, START, HB, REQ, ASSIGN, RESULT, ACK, TAKEOVER, REJOIN, WELCOME, BYE 등)
Worker i → Worker (i mod 10 + 1)	10	CKPT 전송 전용 P2P 연결. 연결 직후 첫 메시지 `PEER i`
Launcher ↔ Worker i	10	L→W `CRASH`, `RESTART` / W→L `READY i`, `REJOINED i`. Worker의 Peer 포트로 접속, 첫 메시지 `LAUNCHER`

Thread 구성 규칙 (Coordinator): Main 1(개수 제외) · Listener 1(select/poll/epoll로 10개 Socket 감시, HB는 직접 lastHB 갱신) · **Dispatcher 정확히 6개** · Failure Detector 1 · (선택) Monitor 1. 실행 중 그 외 Thread를 만들면 안 된다(연결·요청마다 Thread 생성 금지). Worker의 Thread 구성은 자유(권장: Control Recv, Compute, Heartbeat, Peer Server).

1. 계산 작업 — 1 ~ 10^8 사이의 소수 개수

1-1. Job · Task · step — 크기와 예시

Task와 step의 크기 — 예시로 보기

전체 계산은 **1부터 100,000,000(1억)까지**의 수 안에 소수가 몇 개인지 세는 것이다. 이 큰 범위를 두 번 잘라서 다룬다.

단위	개수	크기 (수의 개수)	범위 공식	예시
Job (전체)	1	100,000,000	[1, 100,000,000]	정답: 소수 5,761,455개

Task k	1,000 (0≤k≤999)	100,000 (= 1억 ÷ 1,000)	[k×100,000 + 1, (k+1)×100,000]	Task 0 = [1, 100,000] → 소수 9,592개 Task 1 = [100,001, 200,000] → 8,392개 **Task 3 = [300,001, 400,000]** → 7,863개 Task 999 = [99,900,001, 100,000,000]
step s (Task 안)	Task당 10 (1≤s≤10)	10,000 (= 100,000 ÷ 10)	lo = k×100,000 + (s−1)×10,000 + 1 hi = lo + 9,999	Task 3의 step 1 = [300,001, 310,000] Task 3의 step 10 = [390,001, 400,000] **Task 5의 step 7 = [560,001, 570,000]** → 747개

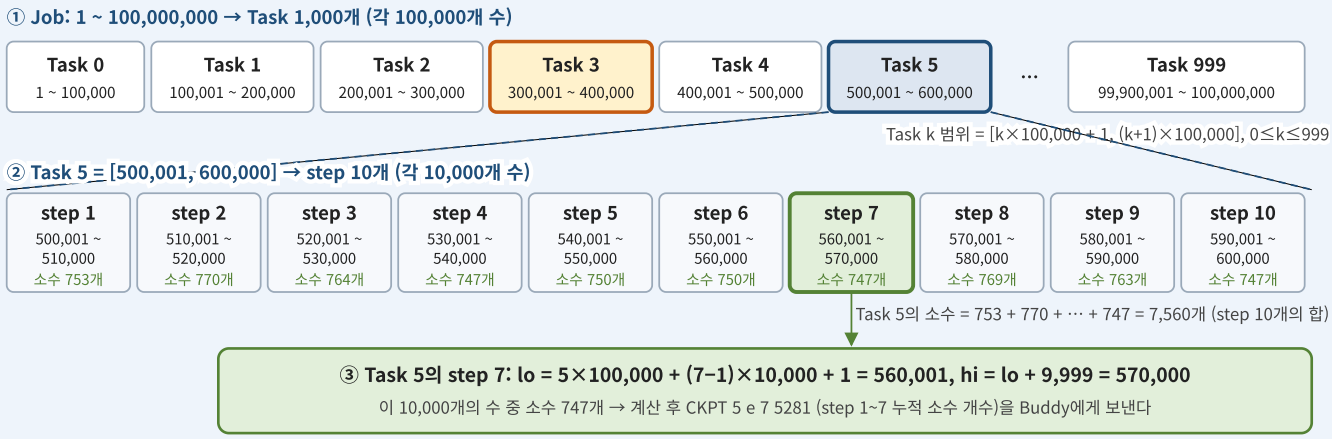


그림 2. 1억 → Task 1,000개(각 100,000) → Task 하나 안의 step 10개(각 10,000). Task 5의 step 7은 [560,001, 570,000]이다

계산 예 — Task 5의 step 7: lo = 5×100,000 + (7−1)×10,000 + 1 = 500,000 + 60,000 + 1 = **560,001**, hi = 560,001 + 9,999 = **570,000**.
역으로 — 수 n이 어느 Task·step에 속하나? k = ⌊(n−1) ÷ 100,000⌋, s = ⌊((n−1) mod 100,000) ÷ 10,000⌋ + 1. 예: n = 565,432 → k = 5, s = 7.
왜 두 번 자르나? Task는 Coordinator가 Worker에게 **나눠 주고 결과를 모으는 단위**이고(Task 결과 1,000개의 합 = 5,761,455), step은 Worker가 Task 안에서 **진행 상황을 Checkpoint로 남기는 단위**다. step s까지 끝낸 뒤 보내는 CKPT의 partial은 "step 1~s의 소수 개수 누적합"이므로, crash가 나도 인계받은 Worker는 step s+1부터 이어서 계산하면 된다(2-3, 2-8). 공식 실행에서 step 하나는 계산 + step-delay 100 ms이므로 Task 하나는 약 1초 걸린다.

1-2. 소수 개수 세는 방법 (권장: 구간 에라토스테네스의 체)

소수는 1과 자기 자신으로만 나누어떨어지는 2 이상의 자연수다(**1은 소수가 아니다**). Worker가 step 하나에서 할 일은 "구간 [lo, hi] (정수 10,000개) 안에 소수가 몇 개인가"를 세는 것이다.

1. **작은 소수 준비 (프로그램 시작 시 한 번):** n이 합성수면 n = a × b (2 ≤ a ≤ b)이고 a ≤ √n이다. 이 과제의 최대 수는 10^8이므로 $\sqrt{10^8}$ = 10,000 이하의 소수만 있으면 된다. 10,000 이하의 소수 1,229개(마지막 9,973)를 기본 에라토스테네스의 체로 구해 `SMALL`에 저장하고 모든 step에서 재사용한다.
2. **구간을 "소수 후보"로 표시:** 길이 hi − lo + 1인 배열 mark를 모두 True로 둔다(mark[n − lo] = n이 소수 후보인가).
3. **각 작은 소수 p의 배수를 지운다:** p × p > hi이면 멈춘다. 지우기 시작점은 **start = max(p × p, lo 이상인 p의 첫 배수)**이고, start, start + p, start + 2p, ⋯ ≤ hi를 False로 바꾼다. p × p보다 작은 p의 배수는 이미 더 작은 소수의 배수로 지워졌고, 이렇게 해야 p 자신(소수)을 지우는 실수도 막는다.
4. **남은 수를 센다:** 끝까지 True로 남은 수가 소수다. 단 lo ≤ 1이면(Task 0의 step 1) 1은 소수가 아니므로 제외한다.

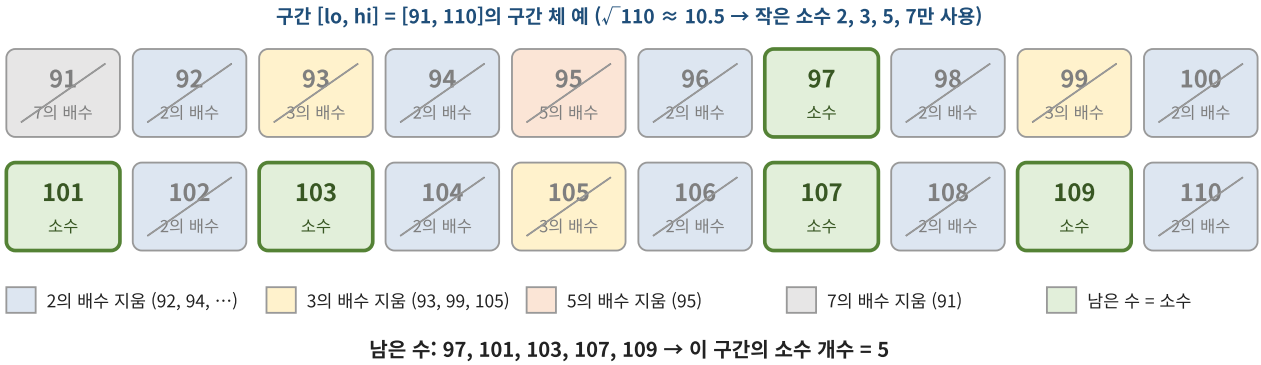


그림 3. 구간 체의 동작 — [91, 110]에서 2·3·5·7의 배수를 지우면 97, 101, 103, 107, 109만 남는다

```
# ① 프로그램 시작 시 한 번: 10,000 이하의 소수 (1,229개, 마지막 9973)
def make_small_primes(limit = 10000):
    is_p = [True] * (limit + 1); is_p[0] = is_p[1] = False
    for p in 2 .. floor(sqrt(limit)):
        if is_p[p]:
            for m in p*p, p*p + p, ⋯, ≤ limit: is_p[m] = False
    return [n for n in 2 .. limit if is_p[n]]
SMALL = make_small_primes()

# ② step s (1≤s≤10)의 구간 [lo, hi]
def step_range(k, s):                                  # 0≤k≤999
    lo = k * 100000 + (s - 1) * 10000 + 1
    return lo, lo + 9999

# ③ 구간 체: [lo, hi] 안의 소수 개수
def count_primes_sieve(lo, hi):
    mark = [True] * (hi - lo + 1)                      # mark[n - lo] = n이 소수 후보인가
    for p in SMALL:
        if p * p > hi: break
        start = max(p * p, ((lo + p - 1) // p) * p)    # lo 이상인 p의 첫 배수와 p*p 중 큰 값
        for m in start, start + p, ⋯, ≤ hi: mark[m - lo] = False
    if lo <= 1: mark[1 - lo] = False                   # 1 제외 (Task 0, step 1에서만)
    return count of True in mark

# ④ Compute Thread가 부르는 함수
def count_primes(k, s):
    lo, hi = step_range(k, s)
    return count_primes_sieve(lo, hi)

# ⑤ Task 하나 계산 (새 Task면 s0=1, partial=0 / 인계면 CKPT 다음 step, CKPT의 partial)
for s in s0 .. 10:
    partial += count_primes(k, s); sleep(step_delay)   # step_delay = 100 ms
    if s < 10: send CKPT k e s partial to Buddy
send RESULT k e partial to Coordinator
# ※ 계산 흐름만 보인 것. crash 처리(crash_gen 비교, send_if_alive)를 넣은 실제 구현은 3-②의 의사코드를 따른다
```

1-3. 검증과 주의할 점

아래 값으로 자신의 소수 세기 함수를 먼저 검증한다(crash 없이 전체 합 5,761,455도 먼저 확인).

Task	step별 누적 소수 개수 (step 1 → 10) — 구현 검증용	결과
0	1229, 2262, 3245, 4203, 5133, 6057, 6935, 7837, 8713, 9592	9,592
152	599, 1204, 1796, 2418, 3031, 3642, 4258, 4872, 5481, 6075	6,075
160	579, 1187, 1794, 2389, 2985, 3560, 4184, 4808, 5399, 5989	5,989
271	561, 1138, 1720, 2319, 2916, 3507, 4069, 4673, 5238, 5802	5,802

확인 항목	기대 값 / 주의

small_primes 개수	1,229개, 마지막 값 9,973
count_primes(0, 1)	1,229 (구간 [1, 10,000] — 1을 소수로 세면 1,230이 되어 틀림)
Task 0 / 152 / 160 / 271	위 표의 step별 누적값과 최종값(9,592 / 6,075 / 5,989 / 5,802)이 모두 일치
[91, 110] 구간	5 (그림 3)
전체 합	Task 1,000개의 결과 합 = 5,761,455 (crash 없이 먼저 확인)
자주 하는 실수	1을 소수로 셈 · 구간 끝 hi를 빼먹음(lo ~ hi 양 끝 포함) · 체에서 p 자신을 지움(시작을 p×p 이상으로) · 매 step마다 small_primes를 다시 계산(느려짐) · 32비트 정수 범위는 문제없음(최대 10^8)

step-delay: 장애를 관찰할 시간을 벌기 위해 step마다 `--step-delay`(기본 100 ms)만큼 쉰다. 계산 자체는 수 ms 안팎이므로 Task 하나 ≈ 1초, 1,000 Task ÷ 10 Worker ≈ 100초 + crash로 인한 지연이다. **정식 실행은 `--step-delay 100` (ms)**으로 한다.

2. 핵심 개념 자세히 보기

2-1. Heartbeat와 Failure Detector

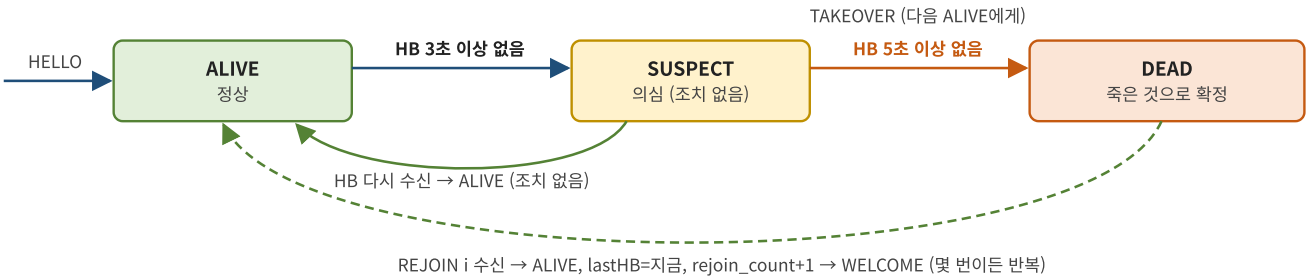


그림 4. Coordinator가 보는 Worker 상태 — DEAD에서 REJOIN으로 몇 번이든 되돌아올 수 있다

지금 − lastHB (sec)	판정	하는 일
< 3	**ALIVE**	없음
3 ≤ … < 5	**SUSPECT**	로그만. Task는 그대로. HB가 오면 ALIVE 복귀
≥ 5	**DEAD**	이 Worker가 owner인 모든 ASSIGNED Task를 다음 ALIVE Worker에게 TAKEOVER

시작 시점: Worker는 START를 받은 뒤에야 HB를 보내므로, Coordinator는 START를 보낸 시각을 모든 Worker의 lastHB 초기값으로 두고 Failure Detector는 **START 이후에** 검사를 시작한다(그 전에 돌리면 Ring 구성 중에 10개가 모두 DEAD가 된다).

Failure Detector가 1초마다 검사하므로 **감지 지연은 5~6초**다. 가상 crash는 연결이 살아 있어 EOF가 없으므로, 감지 단서는 오직 HB의 부재뿐이다. DEAD 상태의 Worker가 보낸 HB는 무시한다(가상 crash 중에는 오지 않지만 안전장치).

2-2. Worker 내부 상태

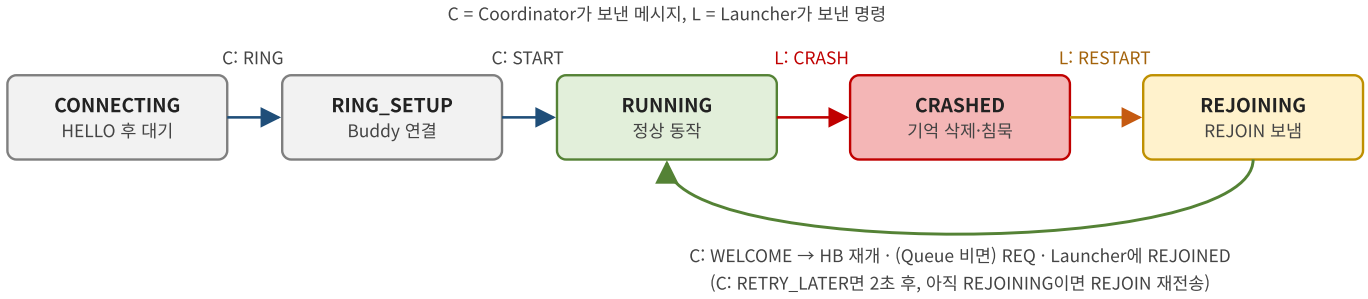


그림 5. Worker 내부 상태 전이 — RUNNING → CRASHED → REJOINING → RUNNING이 반복된다

상태	하는 일	보낼 수 있는 메시지
CONNECTING / RING_SETUP	HELLO 후 RING을 기다리고, Buddy에 연결 후 START를 기다림	HELLO, RING_OK

RUNNING	정상 동작	HB, REQ, RESULT, CKPT, (Launcher에) READY/REJOINED
CRASHED	기억 삭제 후 침묵. 받은 메시지는 버림(단, Launcher의 RESTART와 Coordinator의 BYE는 처리)	**없음**
REJOINING	초기화된 상태로 REJOIN을 보내고 WELCOME을 기다림. 이 동안 들어오는 CKPT는 정상 저장하고, Coordinator 메시지(ACK·REJECT_STALE 등)도 RUNNING과 같은 규칙으로 처리한다	REJOIN (RETRY_LATER면 2초 후, 아직 REJOINING이면 재전송)

2-3. Checkpoint (CKPT)

```
CKPT <task> <epoch> <완료한 step> <지금까지 센 소수 개수>     (task=k: 0≤k≤999, epoch=e: e≥1, step=s: 1≤s≤9)
예) CKPT 152 1 6 3642   ← Task 152를 epoch 1로 계산 중, step 6 완료, 누적 3,642개
```

- Worker i (1≤i≤10)는 step s (1≤s≤9)를 완료할 때마다 항상 **고정 Buddy(i mod 10 + 1)**에게 보낸다. Buddy는 Task별 최신 CKPT만 보관한다. 비교는 **epoch 먼저, epoch가 같으면 step**으로 한다(사전식). 새 CKPT의 (e, s)가 저장된 것보다 크면 덮어쓰고, 작거나 같으면 무시한다.
- 인계받을 때는 Store의 CKPT가 **어느 epoch의 것이든** 그대로 이어서 쓸 수 있다. 소수 개수는 누가 세어도 같기 때문이다(예: W6이 e3로 인계받을 때 W5가 e2로 보낸 CKPT가 있으면 사용).
- Buddy가 crash 중이면 CKPT는 그냥 버려진다(Buddy가 읽고 버림). 보내는 쪽은 신경 쓰지 않는다. 전송 오류가 나도 WARN 후 계속 계산한다.
- CKPT의 partial은 **합계에 더해지지 않는다**. 합계에는 Coordinator가 Commit한 RESULT의 최종값만 들어간다(2-8).
- crash한 Worker는 restart 후 Checkpoint Store가 **비어 있다**. 이후 들어오는 CKPT부터 다시 쌓는다.
- Buddy는 Task가 끝났는지 알 수 없으므로 Store 항목을 지우는 규칙은 없다. 완료된 Task의 CKPT가 남아 있어도 되며(최대 1,000개), 크기가 작으므로 그대로 둔다.

2-4. Takeover와 "다음 ALIVE Worker" 규칙

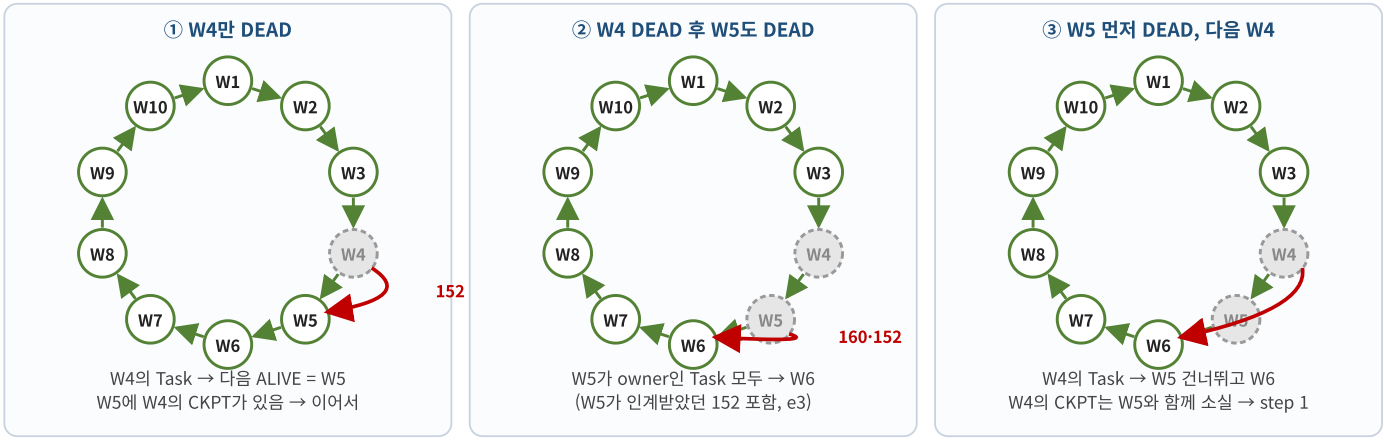


그림 6. 인계 대상 = 링 순서상 다음 ALIVE Worker. 동시 2개 crash에서도 항상 대상이 있다(10개 중 최소 8개 ALIVE)

1. Worker d (1≤d≤10)가 DEAD가 되면 링 순서 d+1, d+2, …(10 다음은 1로 순환)로 보며 처음 만나는 **ALIVE** Worker b (1≤b≤10, b≠d)를 인계 대상으로 정한다. **DEAD와 SUSPECT는 모두 건너뛴다(필수 규칙).** 예: 다음 Worker W5가 이미 DEAD이거나 SUSPECT면 W5를 건너뛰고 W6에게 넘긴다. ALIVE가 하나도 없으면(동시 crash ≤ 2이므로 정상 실행에서는 생기지 않음) 같은 순서로 처음 만나는 SUSPECT Worker에게 넘긴다.
2. 한 번의 검사에서 여러 Worker가 새로 DEAD가 되면 **먼저 모두 DEAD로 표시한 뒤** 번호순으로 인계 대상을 정한다(그래야 방금 죽은 이웃에게 넘기는 일이 없다).
3. d가 owner인 **모든** ASSIGNED Task(d가 이전에 인계받았던 Task 포함)를 epoch+1, owner=b로 바꾸고 `TAKEOVER k e from=d` (0≤k≤999, e≥2)를 보낸다.
4. b는 Checkpoint Store에 k가 있으면 그 다음 step부터, 없으면 step 1부터 계산한다. d의 Buddy가 곧 b라면 대개 CKPT가 있다(①). d의 Buddy가 함께 crash했다면 CKPT는 없다(③).
5. b가 사실은 crash 직후(아직 DEAD 판정 전)여서 TAKEOVER를 버리더라도 괜찮다. b가 DEAD 판정될 때 그 Task가 다시 다음 ALIVE에게 넘어가기 때문이다(②, epoch는 한 번 더 증가).

2-5. Commit · Epoch · 이중 Commit 개념

용어	뜻	이 과제에서
Commit	Coordinator가 어떤 Task의 결과를 **최종 확정**하는 것. 확정된 Task는 DONE이 되고 결과가 합계에 들어간다. 한 번 확정된 Task는 다시 바뀌지 않는다	유효한 RESULT → Task DONE, result 저장, ACK 전송
commit_count	Task가 Commit된 횟수	종료 시 1,000개 모두 정확히 1이어야 한다
owner	지금 그 Task를 맡고 있는 Worker	ASSIGN 때 요청자, TAKEOVER 때 인계 대상
Epoch (e≥1)	Task의 담당자가 바뀐 횟수를 나타내는 번호. **Fencing Token**이라고도 한다	ASSIGN 때 1, TAKEOVER 때마다 +1
이중 Commit	같은 Task가 두 번 확정되어 결과가 두 번 더해지는 오류. 값이 맞아도 전체 합계가 틀린다	반드시 0건 (HW#2의 이중예약에 해당)
REJECT_STALE	유효하지 않은(옛 담당자의, 또는 이미 확정된 Task의) 결과를 거부하는 응답	DONE·EPOCH·DEAD 이유와 함께 전송

왜 필요한가? Worker가 crash 직전에 보낸 RESULT가 네트워크에서 늦게 도착할 수 있다. 그사이 Coordinator는 그 Worker를 DEAD로 판정하고 Task를 다른 Worker에게 넘겼을 수 있다. 결국 **같은 Task의 결과가 두 번** 들어온다. Coordinator는 "지금 이 Task의 정당한 담당자가 보낸 결과인가?"를 가려야 하고, 그 기준이 **epoch와 owner**다.

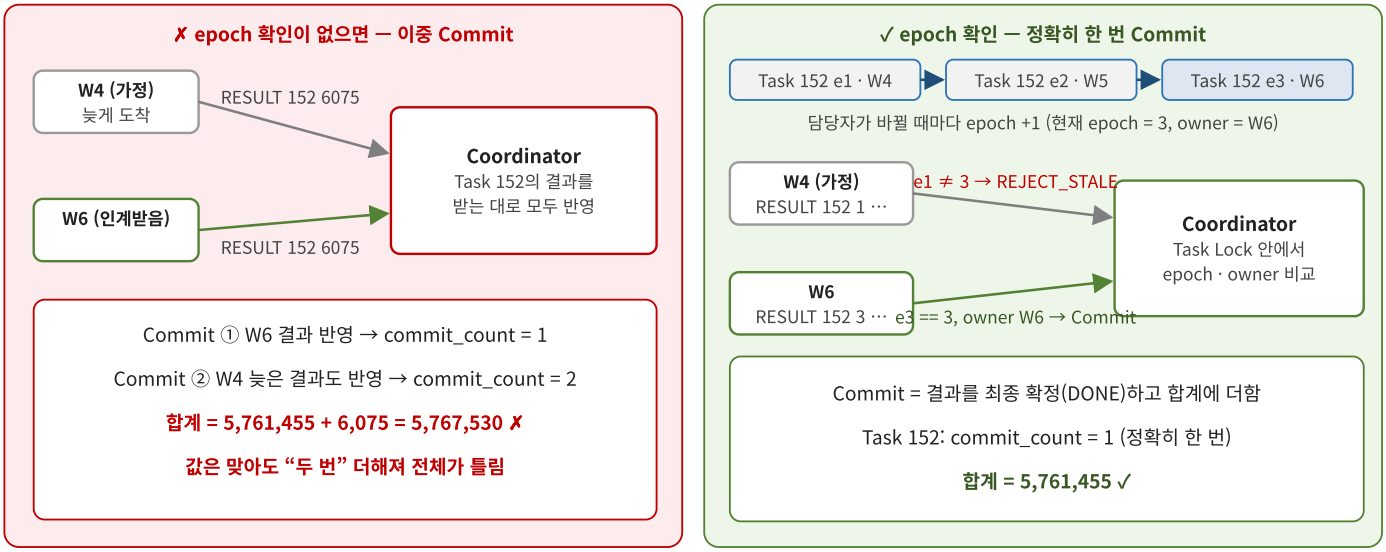


그림 7. 왼쪽: epoch 확인이 없으면 늦게 도착한 옛 결과까지 반영되어 이중 Commit. 오른쪽: Task Lock 안에서 epoch·owner를 비교해 정확히 한 번 Commit (W4의 늦은 RESULT는 규칙을 보여 주기 위한 **가정**: W4가 step 10까지 끝내고 RESULT를 보낸 직후 crash한 경우)

epoch는 호텔 카드키 번호와 같다. 방(Task)의 주인이 바뀔 때마다 새 카드키를 발급하고 옛 카드키는 무효로 만든다. 옛 카드키(옛 epoch)로 온 결과는 문이 열리지 않는다(REJECT_STALE). 그리고 이 비교와 DONE 표시는 반드시 **그 Task의 Lock 안에서 한 번에** 해야 한다(아래 상자).

> **확인과 변경은 반드시 한 Lock 안에서**
> RESULT 처리에서 "DONE 아님 · epoch 일치 · owner 일치"를 Lock 밖에서 확인한 뒤 Lock을 잡아 DONE으로 바꾸면, 그 사이 FD가 Task를 다른 Worker에게 넘기고 그 Worker의 RESULT도 통과해 **commit_count = 2**가 될 수 있다. 같은 이유로 REQ의 "DEAD 아님 확인 + 배정"은 Worker Table Lock 안에서 한 번에 해야 한다(분리하면 DEAD Worker에게 배정된 "고아 Task"가 남아 999/1000에서 멈춘다).

2-6. Task 상태 전이

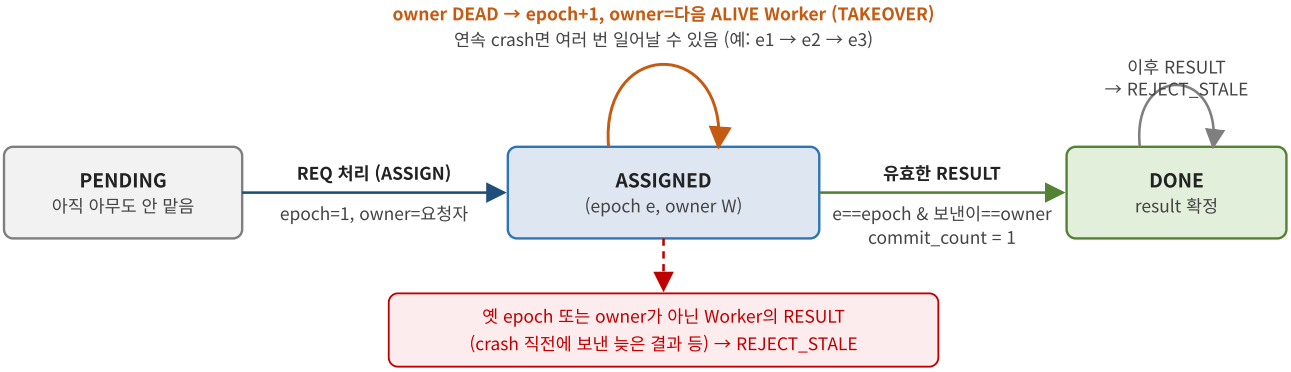


그림 8. Task 상태 전이 — DONE으로 가는 길은 "유효한 RESULT" 하나뿐, epoch는 인계될 때마다 증가

DONE으로 가는 길은 "유효한 RESULT" 하나뿐이다. 이웃한 Worker가 연달아 crash하면 한 Task가 여러 번 인계되어 epoch가 1 → 2 → 3처럼 증가할 수 있다(5장 Step 4).

2-7. Rejoin

- Launcher의 `RESTART` → Worker i (1≤i≤10)는 모든 상태를 초기화했는지 확인하고 `REJOIN i`를 보낸다.
- Coordinator: i가 **DEAD**면 ALIVE, lastHB=지금, rejoin_count[i]+1 (1≤i≤10) → `WELCOME`. 아직 ALIVE·SUSPECT면(= DEAD 판정 전) `RETRY_LATER`.
- Worker: WELCOME → RUNNING, HB 재개, 로컬 Queue가 비어 있으면 `REQ i` → **새 Task**를 받는다. Launcher에게 `REJOINED i`를 보낸다.
- 재합류한 Worker는 이전 Task를 전혀 모른다. 이전 Task는 DEAD 때 이미 다른 Worker에게 넘어갔다. 늦게 도착한 ACK·REJECT_STALE이 모르는 Task에 대한 것이면 무시한다.

2-8. 전체 합계는 어떻게 계산되나? — 결과가 모이는 과정

Worker가 step마다 다음 Worker에게 보내는 CKPT는 **백업**일 뿐, 결과를 전달하는 길이 아니다. 결과가 합계로 모이는 길은 하나뿐이다: **Task를 끝낸 Worker가 Coordinator에게 보내는 RESULT**. Coordinator는 이 RESULT를 검사해 유효할 때만 **Commit**하여 Task Table의 `result[k]`에 기록하고, 마지막에 1,000개를 모두 더한다.

메시지	어디로	담긴 값	합계에 들어가나?
CKPT k e s partial	Worker i → Buddy(i) (Coordinator를 거치지 않음)	Task k의 step 1~s까지의 **중간 누적값** (1≤s≤9)	**아니오 — 인계받은 Worker가 이어서 계산할 출발점**
RESULT k e v	Worker → Coordinator	Task k의 step 1~10 전체 **최종값** v	**유효하면 예 (Commit) / 아니면 거부**

(1) 한 Task의 결과가 만들어지는 과정 — crash와 인계가 있어도 값은 이어진다

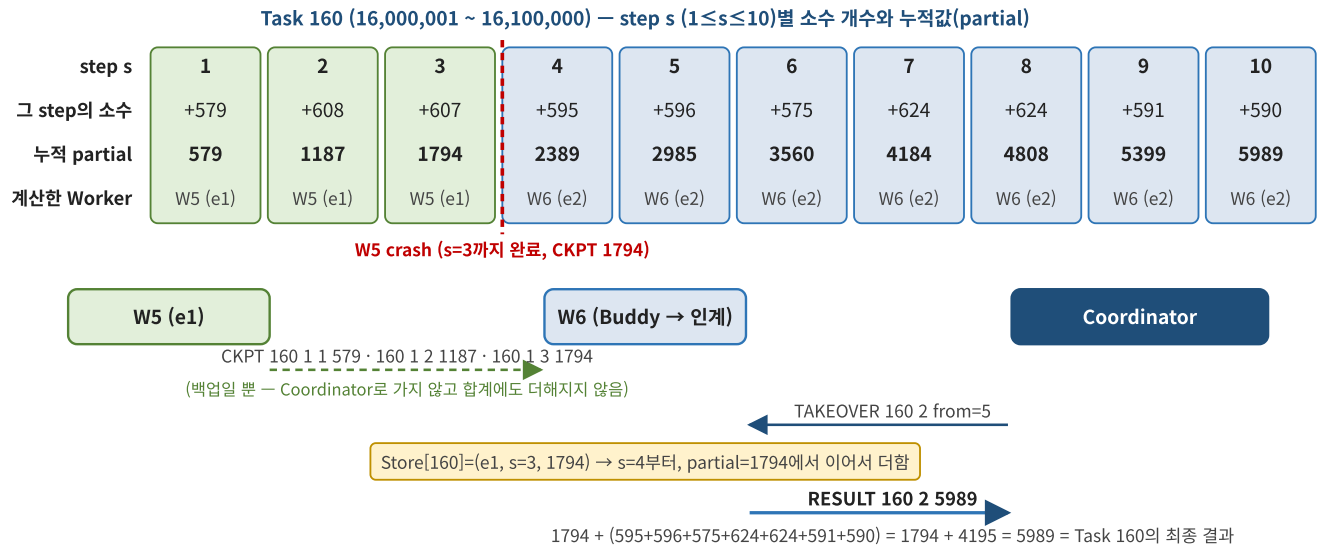


그림 9. Task 160: W5가 s=1~3을 계산(누적 1794)하고 crash → W6이 CKPT의 1794에서 s=4부터 이어 계산 → 최종 5989를 RESULT로 Coordinator에 보낸다

- **누적(partial)의 의미:** partial은 "이 Task에서 지금까지 센 소수 개수"다. step s를 끝내면 partial += (그 step의 소수 개수).
- **인계 시:** 인계받은 Worker는 CKPT의 (s, partial)에서 시작해 s+1 ~ 10을 더한다. CKPT가 없으면 s=1, partial=0에서 시작한다 (Task 152, 6075). 어느 경우든 소수 개수는 누가 세어도 같으므로 **최종값은 항상 같다.**
- **crash한 Worker가 세던 값**(W5의 1794)은 그 자체로는 어디에도 더해지지 않는다. 인계받은 Worker의 계산 안에 "출발점"으로만 포함된다.

(2) Coordinator의 결과 장부와 합계

Dispatcher가 처리한 RESULT (도착 순서)

① **RESULT 0 1 9592 (W1)**
e1=1, owner W1 → Commit → result[0]=9592

② **RESULT 160 2 5989 (W6)**
e2=2, owner W6 → Commit → result[160]=5989

③ **RESULT 152 3 6075 (W6)**
e3=3, owner W6 → Commit → result[152]=6075

④ **(가정) RESULT 152 1 6075 (W4의 늦은 결과)**
W4가 DEAD면 DEAD, 아니면 이미 DONE → REJECT_STALE — 합계 +0

⑤ **RESULT 160 2 5989 (같은 결과가 또 도착했다고 가정)**
이미 DONE → REJECT_STALE (DONE) — 합계 변화 없음

(참고) CKPT 160 1 3 1794
Coordinator로 오지 않음 → Worker 간 백업 — 합계와 무관

Coordinator의 Task Table (결과 장부, 발췌)

k	state	epoch	owner	result	commit
0	DONE	1	W1	9592	1
…	…	…	…	…	…
152	DONE	3	W6	6075	1
160	DONE	2	W6	5989	1
271	ASSIGNED	1	W4	—	0
…	…	…	…	…	…
999	PENDING	0	—	—	0

Commit될 때만 result가 채워지고 commit_count가 1이 된다

종료 시 Main Thread의 VERIFY: 합계 = Σ result[k] (0≤k≤999, 모두 DONE) = 5,761,455
그리고 Σ commit_count[k] = 1,000 (모든 Task가 정확히 1) — 거부된 RESULT와 CKPT는 합계에 들어가지 않는다

그림 10. Coordinator는 유효한 RESULT만 Commit해 result[k]에 기록한다. 늦게 온 옛 결과(④)와 중복 결과(⑤)는 거부되어 합계에 영향을 주지 않는다

#	도착한 RESULT (상황)	Task Lock 안에서의 판정	Task Table 변화 / 합계 영향
①	RESULT 0 1 9592 — 정상 흐름(W1)	DONE 아님, e=1=epoch, owner=W1	DONE, result[0]=9592, commit_count=1 → **+9,592**
②	RESULT 160 2 5989 — W5 crash 후 W6이 CKPT로 이어서 완료	e=2=epoch, owner=W6	result[160]=5989 → **+5,989**
③	RESULT 152 3 6075 — 두 번 인계 (W4→W5→W6), CKPT 없이 처음부터	e=3=epoch, owner=W6	result[152]=6075 → **+6,075**

④	(가정) RESULT 152 1 6075 — W4가 step 10까지 끝내고 보낸 직후 crash했다면, 늦게 도착한 옛 결과(stale)	W4가 아직 DEAD면 DEAD, rejoin 후면 이미 DONE (또는 e=1≠3)	REJECT_STALE → **+0** (이것을 받으면 이중 Commit)
⑤	RESULT 160 2 5989 — 같은 결과가 한 번 더 도착했다고 가정	이미 DONE	REJECT_STALE → **+0**
⑥	RESULT 271 1 5802 — W4가 rejoin 후 새로 받은 Task	e=1=epoch, owner=W4	result[271]=5802 → **+5,802**

(3) 합계를 계산하는 방법 (둘 중 하나, 또는 둘 다)

방법	구현	주의
A. 종료 시 한 번에 (권장)	Main Thread가 1,000개 DONE을 확인한 뒤 Σ result[k] (0≤k≤999)를 계산하고, Σ commit_count[k] = 1,000인지도 함께 확인 → VERIFY 로그	모든 Dispatcher가 멈춘 뒤 읽으므로 Lock 경합이 없다
B. Commit할 때마다 누적	Commit 직후 `total += v`, `done_count += 1` (진행 상황 POOL 로그에 활용 가능)	여러 Dispatcher가 동시에 더하므로 **total도 Mutex로 보호**. 종료 시 A의 값과 같아야 한다

참고: ④와 ⑤는 거부 규칙을 보여 주기 위한 **가정**이다. seed=42 실행(5장 Step 4)에서 W4는 152의 step 6에서 crash했으므로 e1의 RESULT는 실제로 보내지 않았고, 그래서 6-3 결과 예시의 REJECT_STALE는 0이다.

정리: 합계가 5,761,455가 되려면 ① 1,000개 Task가 모두 DONE이고(빠짐없음), ② 각 Task가 정확히 한 번 Commit되어야 한다(중복 없음). crash·인계·stale 결과가 어떻게 섞여도, "유효한 RESULT만, Task Lock 안에서, 한 번만 Commit"하면 이 두 조건이 지켜진다.

2-9. Task 흐름 따라가기 — 최초 배정 → 계산 → 반납, 그리고 crash 시 인계

지금까지의 개념을 하나의 Task 흐름으로 이어서 본다. (1) Task가 처음에 어떻게 Worker에게 가는지, (2) Worker가 받은 Task를 어떻게 계산하고 RESULT로 반납하는지, (3) Worker가 crash하면 하던 일이 어떻게 Buddy에게 넘어가고, Buddy가 **자기가 하던 Task와 인계받은 Task**를 어떻게 둘 다 끝내는지를 순서대로 설명한다.

(1) 최초 배정 — Coordinator는 Task를 "밀어 넣지" 않고, Worker가 "요청(REQ)"해야 준다

- **시작 상태:** Coordinator는 시작할 때 Task Table 1,000칸을 모두 `PENDING`(epoch 0, owner 없음)으로 만들고, PENDING Queue에 Task 번호 0, 1, 2, …, 999를 **오름차순**으로 넣어 둔다.
- **첫 REQ:** Worker는 START를 받는 순간 로컬 Task Queue가 비어 있으므로 `REQ i`를 **한 번** 보낸다. 10개의 REQ는 Listener를 거쳐 Request Queue에 **도착한 순서대로** 쌓인다.
- **배정:** Dispatcher는 REQ를 하나 꺼내 Worker Table Lock 안에서 ① 그 Worker가 DEAD가 아닌지(ALIVE 또는 SUSPECT) 확인하고 ② PENDING Queue의 맨 앞 번호 k를 꺼내 ③ Task[k]를 `ASSIGNED, owner=i, epoch=1`로 바꾼 뒤, Lock을 풀고 `ASSIGN k 1`을 보낸다(3-①). 따라서 **Worker Table Lock 안에서 먼저 poll()한 REQ가 더 작은 번호**를 받는다. Dispatcher 6개가 동시에 꺼내므로 이 순서는 **대개** 도착 순서와 같지만 항상 같지는 않다.
- **고정 배정이 아니다:** "W1에게 Task 0, W2에게 Task 1…"처럼 미리 정하지 않는다. 누가 어떤 번호를 받는지는 REQ 도착 순서로 정해지며 실행마다 달라진다. Dispatcher 6개가 동시에 REQ를 처리해도 poll()이 Lock 안에서 하나씩 일어나므로 같은 번호가 두 번 나가지 않는다.

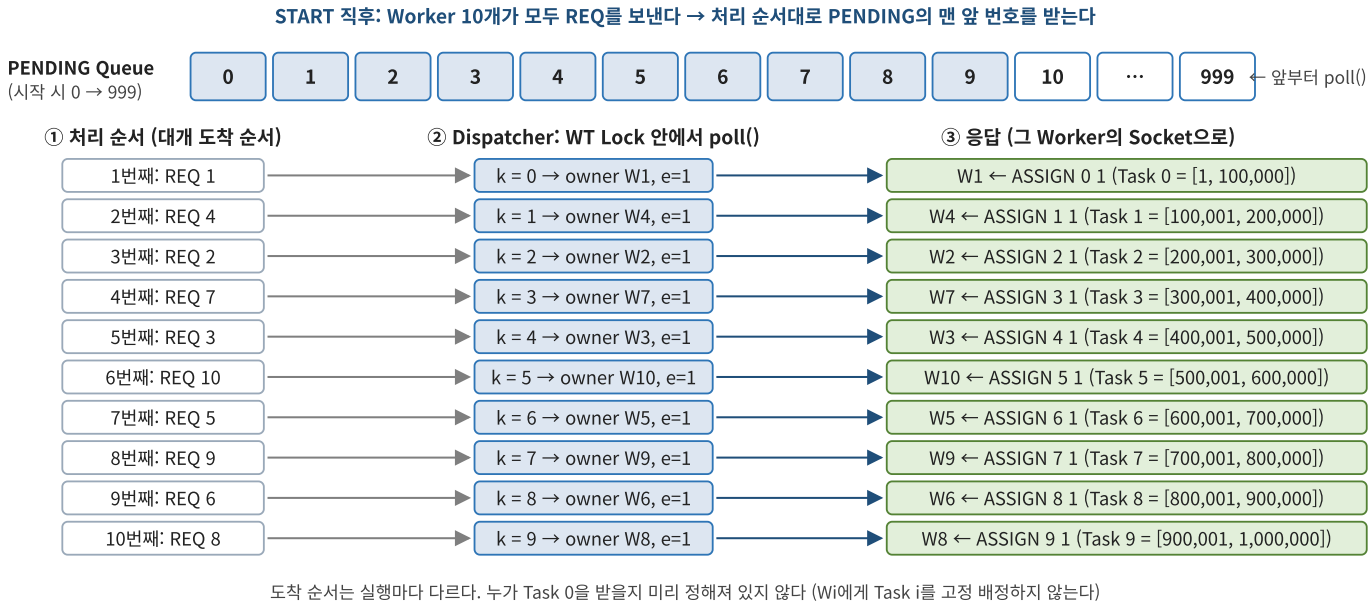


그림 11. 최초 배정 예 — REQ 처리 순서 W1, W4, W2, W7, W3, W10, W5, W9, W6, W8 → Task 0 ~ 9가 그 순서대로 배정된다

그다음부터 (pull 방식): Worker는 받은 Task를 다 끝내 로컬 Queue가 비면 다시 `REQ i` 를 보내고, 그 순간 PENDING의 맨 앞 번호를 받는다. 처음 10개가 거의 동시에 시작했으므로 두 번째 배정(Task 10 ~ 19)도 대체로 같은 순서로 나가지만, 조금 빨리 끝낸 Worker가 먼저 받는다. 이렇게 빨리 끝낸 Worker가 더 많이 가져가 부하가 저절로 나뉜다.

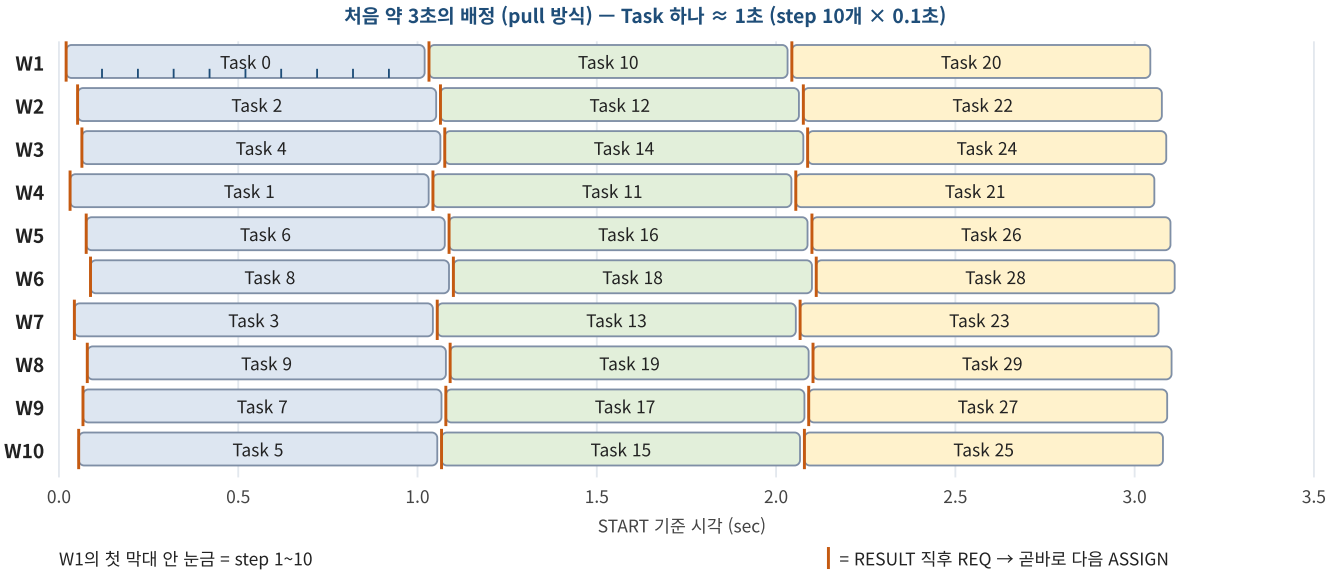


그림 12. 처음 약 3초 동안의 배정 — 각 Worker는 Task 하나(≈1초)를 끝낼 때마다 REQ를 보내 다음 번호를 받는다

(2) Worker 안에서 — 받은 Task를 계산하고 RESULT로 반납하기 (예: W1이 Task 0을 받은 경우)

#	누가	하는 일 (Task 0, epoch 1, Buddy = W2)
1	W1 Control Recv	`ASSIGN 0 1` 수신 → 로컬 Queue에 **(k=0, e=1, 시작 step=1, partial=0)**을 넣고 notify. REQ 대기 플래그를 내린다
2	W1 Compute	Queue에서 꺼낸다. step_range(0, s)로 구간을 구해 step 1부터 차례로 센다: step 1 = [1, 10,000] → 1,229개, step 2 = [10,001, 20,000] → +1,033 …
3	W1 Compute → W2	step s (1≤s≤9)를 끝낼 때마다 누적값을 `CKPT 0 1 s partial` 로 Buddy W2에게 보낸다. 예: `CKPT 0 1 1 1229` , `CKPT 0 1 2 2262` , …, `CKPT 0 1 9 8713`
4	W2 Peer Server	Store[0]을 최신 값으로 덮어쓴다: (e1, s=1, 1229) → (e1, s=2, 2262) → … → (e1, s=9, 8713). W2는 이 값을 계산에 쓰지 않고 **보관만** 한다
5	W1 Compute → C	step 10까지 끝내면 CKPT 대신 `RESULT 0 1 9592` 를 Coordinator로 보낸다

6	Coordinator Dispatcher	Task[0] Lock 안에서 DONE 아님, e=1=epoch, 보낸 Socket = owner W1 확인 → **Commit**: DONE, result[0]=9592, commit_count=1 → `ACK 0`
7	W1	로컬 Queue가 비었으므로 `REQ 1` → 그 시점 PENDING의 맨 앞(예: Task 10) → `ASSIGN 10 1` → 1번부터 반복

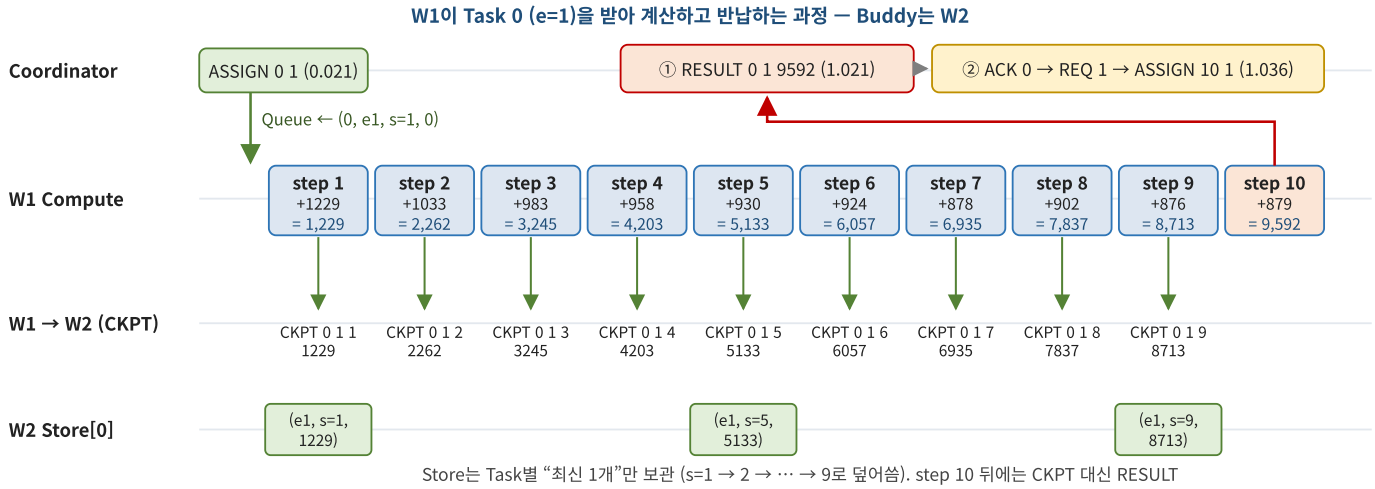


그림 13. W1의 Task 0 처리 — step마다 누적값을 Buddy W2에 CKPT로 백업하고, step 10 뒤에 RESULT로 반납한다(괄호 안은 START 기준 시각, sec)

시각 (sec)	W1	W2 Store[0]	Coordinator
0.021	`ASSIGN 0 1` 수신	—	Task 0: ASSIGNED, e1, owner W1
0.121	step 1 완료 (1,229) → CKPT	(e1, s=1, 1229)	—
0.521	step 5 완료 (5,133) → CKPT	(e1, s=5, 5133)	—
0.921	step 9 완료 (8,713) → CKPT	(e1, s=9, 8713)	—
1.021	step 10 완료 (9,592) → `RESULT 0 1 9592`	그대로	Commit → result[0]=9592 → `ACK 0`
1.036	`REQ 1`	—	`ASSIGN 10 1`

이 예는 ACK를 받은 뒤 REQ를 보내는 방식이다(1.036). 3-②의 의사코드처럼 RESULT 직후 곧바로 REQ를 보내도 된다(3-⑤, 둘 다 허용). step 하나 ≈ 0.1초(계산 수 ms + step-delay 100 ms)이므로 Task 하나 ≈ 1초다(계산이 느린 환경에서는 조금 더 걸린다). crash가 없으면 Buddy의 Store에 쌓인 CKPT는 끝내 쓰이지 않는다. CKPT는 "crash가 났을 때만" 쓰는 보험이다.

(3) crash 시 — Buddy가 하던 일과 인계받은 일을 둘 다 끝내기 (seed=42 스케줄 #6: W7 crash)

상황: W7은 Task 690 (e1)을 계산하던 중 step 4까지 끝내고 `CKPT 690 1 4 2196`을 Buddy W8에 보냈다. 67.0초에 Launcher의 CRASH로 W7이 가상 crash한다(step 5 계산은 사라짐). 그 순간 W8은 자기 Task들을 계산하고 있고, 72.0초에는 **자기 Task 736 (e1)**의 step 2까지 끝내고 step 3을 계산 중이다.

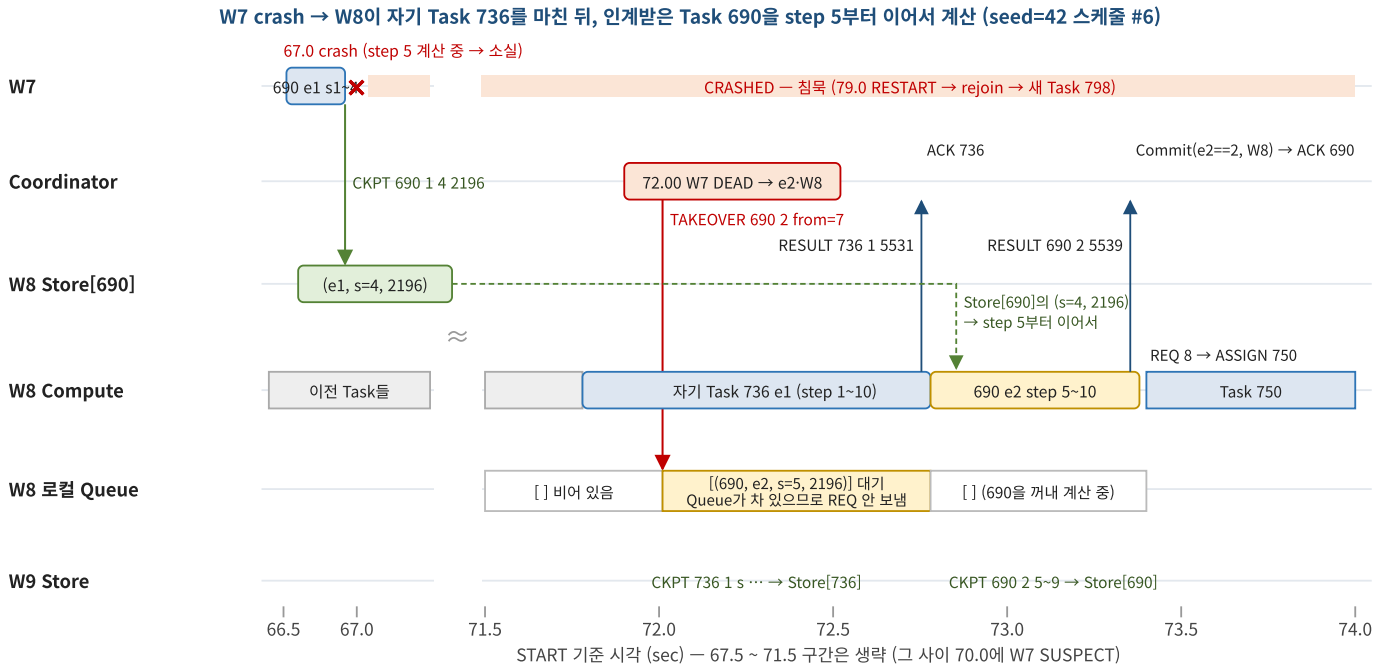


그림 14. W7 crash → 72.00 DEAD → W8에게 TAKEOVER 690 2. W8은 진행 중이던 736을 끝내 반납한 뒤, Store[690]의 (s=4, 2196)부터 690을 이어서 계산해 반납한다

시각 (sec)	W7	Coordinator	W8 (Buddy, 인계 대상)
66.52	`ASSIGN 690 1` 받고 계산 시작	690: e1, owner W7	(자기 Task 계산 중)
66.92	step 4 완료(2,196) → `CKPT 690 1 4 2196`	—	Store[690] = (e1, s=4, 2196)
66.95	마지막 `HB 7`	lastHB[7] = 66.95	—
67.00	CRASH → Queue·Store 비움, 침묵 (연결 유지)	모름 (EOF 없음)	계속 계산. W8의 Buddy인 W9에게 자기 CKPT 전송
70.00	침묵	W7 SUSPECT (3.05초)	—
71.78	침묵	`ASSIGN 736 1` → W8	자기 Task 736 시작
72.00	침묵	W7 DEAD → 다음 ALIVE = W8 → 690: **e2, owner W8** → `TAKEOVER 690 2 from=7`	736 step 2 완료(1,059), step 3 계산 중
72.01	—	—	Control Recv: Store[690] = (s=4, 2196) → Queue에 **(690, e2, 시작 step 5, 2196)** 추가. Compute는 736을 **계속** 계산. Queue가 비어 있지 않으므로 REQ는 보내지 않는다
72.78	—	736 Commit → `ACK 736`	736 step 10 완료 → `RESULT 736 1 5531`
72.78~73.38	—	—	Queue에서 690을 꺼내 step 5 ~ 10 계산: 2196 + 561 + 560 + 549 + 566 + 544 + 563. step 5~9마다 `CKPT 690 2 s partial`을 **W9**에게 (epoch 2)
73.38	—	e2 == epoch 2, 보낸이 W8 == owner → Commit(73.39), result[690] = 5539 → `ACK 690`	`RESULT 690 2 5539`
73.39	—	`ASSIGN 750 1`	Queue가 비었으므로 `REQ 8`

79.00~	RESTART → `REJOIN 7` → `WELCOME` → `REQ 7` → `ASSIGN 798 1` (690은 모름)	W7 ALIVE, rejoin_count[7]=1	—

이 예에서 확인할 점

1. **인계받은 Task는 Queue에 "한 줄"로 들어갈 뿐이다.** TAKEOVER는 Control Recv가 받아 (k, e, 시작 step, partial)을 로컬 Queue에 넣기만 한다. Compute Thread는 하나이므로 두 Task를 동시에 계산하지 않는다. 이 예처럼 **진행 중인 Task(736)를 끝낸 뒤** 인계 Task(690)로 넘어가는 방법을 권장한다. 인계 Task를 먼저 해도 되지만(순서는 자유), 그렇게 하려면 멈춘 Task의 (step, partial)을 Queue에 다시 넣어야 한다.
2. **시작점은 Buddy의 Store가 정한다.** Store[690] = (e1, s=4, 2196)이므로 step 5부터, partial 2196에서 이어 센다. Store에 없으면 step 1, partial 0부터다(5장 Step 4). 어느 쪽이든 최종값은 5,539로 같다.
3. **RESULT와 CKPT에는 새 epoch(2)를 쓴다.** W8이 보내는 `CKPT 690 2 s …` 는 W8의 Buddy인 W9에게 간다. W8까지 crash 해도 W9가 690을 이어받을 수 있다.
4. **REQ는 Queue가 빌 때만 보낸다.** W8은 736과 690을 모두 끝낸 73.39초에야 REQ를 보낸다.
5. **합계에 들어가는 것은 Commit된 RESULT뿐이다.** result[736] = 5531, result[690] = 5539가 각각 한 번씩 기록된다. W7의 2,196은 W8 계산의 출발점으로만 쓰인다(2-8).
6. **지표:** 감지 지연 = 72.00 − 66.95 = 5.05초, 복구 시간 = 73.39 − 72.00 ≈ 1.4초(6-3 결과 예시), CKPT 절약 4/10 step. crash한 W7은 rejoin 후 690을 모른 채 새 Task 798을 받는다.

3. 시스템 구성 요소 (상세 요구사항)

원격 Coordinator 1개, 로컬 Worker 10개, 로컬 Chaos Launcher 1개로 구성한다. 모든 통신은 Socket(TCP)으로 구현한다. 프로그래밍 언어는 자유이며, 해당 언어의 Thread, Mutex, Condition Variable(또는 동등한 블로킹 대기/통지 기능)을 직접 사용한다.

① Coordinator (원격 호스트)

필수 사항

- Task Table[0..999] (state · owner · epoch · result · commit_count)와 Worker Table[1..10] (ALIVE/SUSPECT/DEAD · lastHB · rejoin_count · Socket)을 둔다. 시작 시 모든 Task는 PENDING, PENDING Queue에 0~999를 오름차순으로 넣는다.
- **Task별 Mutex**(Task Table 전체를 Lock 1개로 보호 금지), Worker Table Lock 1개. 여러 Lock을 잡을 때는 항상 **Worker Table Lock → Task Lock(번호 오름차순)**. Lock을 잡은 채 send하지 않는다.
- Listener 1개가 연결 수락과 10개 Socket 감시(select/poll/epoll 등), 줄 단위 파싱을 맡는다. HB는 Listener가 직접 lastHB를 갱신하고(Queue 지연으로 인한 오판 방지), 나머지는 Request Queue에 넣는다.
- **Dispatcher 6개(고정)**가 Request Queue에서 메시지를 꺼내 아래 "Coordinator 판정 규칙"대로 처리하고, 응답은 보낸 Worker의 Socket으로 직접 보낸다(Worker Socket마다 송신 Lock).
- Failure Detector 1개가 START 이후 1초마다 lastHB를 검사해 SUSPECT/DEAD를 판정하고 TAKEOVER를 수행한다. 인계 대상은 다음 **ALIVE** Worker(DEAD·SUSPECT 건너뜀, ALIVE가 없으면 첫 SUSPECT).
- 모든 대기(Request Queue가 빔, 종료 대기 등)는 Condition Variable로 구현한다(while로 조건 재확인). 1,000번째 Commit을 한 Dispatcher가 종료 CondVar로 Main을 깨우고, Main이 VERIFY · BYE · join을 수행한다.
- 5초마다 POOL 로그(Queue 길이, PENDING/ASSIGNED/DONE 수, ALIVE 목록)를 남긴다(Listener의 select 타임아웃 또는 선택 Monitor Thread).

Coordinator 판정 규칙 (Dispatcher · Listener · FD 공통, 모든 조 동일)

- **보낸 Worker의 판단:** 메시지 안의 번호가 아니라 **메시지가 도착한 Socket**으로 판단한다. HELLO 때 "Socket ↔ Worker 번호"를 Worker Table에 등록해 두고, 번호가 없는 `RESULT k e v` 도 이 Socket으로 보낸 Worker(owner 비교 대상)를 안다. HB i · REQ i 등의 번호는 로그 · 검증용이다.
- **기동:** HELLO 10개가 모이면 모두에게 `RING j ip port` (j = i mod 10 + 1), RING_OK 10개가 모이면 `START` (시각 0). 이미 등록된 번호 · 1~10 밖 번호 · START 이후의 HELLO는 무시(WARN). 그 전에는 REQ에 응답하지 않는다.
- **HB (Listener가 직접):** ALIVE · SUSPECT면 lastHB 갱신(SUSPECT는 ALIVE로 복귀), DEAD면 무시.
- **REQ i** (Worker Table Lock 안에서 확인과 배정을 한 번에): ① i가 DEAD면 무시(WARN) ② PENDING Task가 있으면 맨 앞 k를 꺼내 Task Lock → ASSIGNED, owner = i, epoch = 1 → Lock 해제 후 `ASSIGN k 1` (SUSPECT여도 배정) ③ PENDING이 비었으면 `NO_TASK`.

- **RESULT k e v** — 위에서부터 순서대로 판정한다. ①은 Worker Table Lock으로 상태를 읽고 푼 뒤, ②~④는 Task[k] Lock 안에서 확인과 변경을 한 번에 한다(Lock을 겹쳐 잡지 않으므로 Lock 순서 문제 없음): ① 보낸 Worker가 DEAD → `REJECT_STALE k DEAD` ② Task k가 이미 DONE → `REJECT_STALE k DONE` ③ e ≠ 현재 epoch 또는 보낸 Worker ≠ owner → `REJECT_STALE k EPOCH` ④ 그 외 → **Commit**(DONE, result[k] = v, commit_count + 1) → Lock 해제 후 `ACK k`. 1,000번째 Commit이면 종료 CondVar로 Main을 깨운다.
- **REJOIN i:** i가 DEAD면 ALIVE, lastHB = 지금, rejoin_count + 1 → `WELCOME` / 아직 ALIVE · SUSPECT면(DEAD 판정 전) `RETRY_LATER`.
- **DEAD 처리 (FD):** Worker Table Lock 안에서 새로 DEAD인 Worker를 먼저 모두 DEAD로 표시 → 번호순으로 다음 ALIVE b 결정 → owner = d인 모든 ASSIGNED Task를 Task 번호 오름차순으로 하나씩 Lock 잡고 epoch + 1, owner = b → Lock을 모두 푼 뒤 `TAKEOVER k e from=d` 전송(2-4).

② Worker (로컬, 10개 프로세스)

필수 사항

- 시작 시 Peer 포트(7000 + i)를 먼저 열고 Coordinator에 접속한다(접속이 거절되면 0.5초 후 재시도). 접속 후 `HELLO i ip port`. RING을 받으면 Buddy의 Peer 포트에 연결해 `PEER i`를 보낸 뒤 `RING_OK i`. Peer 포트로는 선행 Worker의 CKPT 연결과 Launcher 제어 연결(첫 메시지 `LAUNCHER`)이 들어온다.
- RUNNING일 때 1초마다 HB, 로컬 Queue가 비었을 때만 REQ(응답 전 재전송 금지). Task는 step 1 → 10 순서로 계산하고 step 1~9마다 CKPT를 Buddy에게, step 10 뒤 RESULT를 Coordinator에게 보낸다.
- **START**를 받으면 RUNNING이 되어 HB를 시작하고 REQ를 보낸다. Launcher에게는 "START 수신"과 "Launcher 제어 연결됨"이 **모두** 충족된 순간(둘 중 늦은 쪽) `READY i`를 보낸다(Launcher 연결이 START보다 늦어도 READY는 반드시 전달).
- **NO_TASK**를 받으면 no_task 플래그를 세우고 이후 REQ를 보내지 않는다(TAKEOVER로 받은 Task를 끝낸 뒤에도). TAKEOVER · BYE만 기다린다. crash하면 이 플래그도 지워지므로 rejoin 후에는 REQ를 한 번 보낸다(다시 NO_TASK가 온다).
- **ASSIGN k e**를 받으면 로컬 Queue에 (k, e, 시작 step 1, partial 0)을 넣고 **REQ 대기 플래그(requested)를 내린 뒤** Queue CondVar를 notify한다. 이 플래그를 내리지 않으면 다음 REQ를 영영 보내지 못한다.
- **TAKEOVER k e from=d**를 받으면 Store에 k가 있으면 (k, e, 저장 step + 1, 저장 partial), 없으면 (k, e, 1, 0)을 로컬 Queue에 넣고 Queue CondVar를 notify한다(2-4, 2-9). **ACK · REJECT_STALE**는 로그 · 통계용이며 모르는 Task면 무시한다.
- **CRASH를 받으면** 로컬 Queue · 진행 중 Task · Checkpoint Store · 플래그를 모두 지우고, **RESTART를 받을 때까지 어떤 메시지도 보내지 않는다**(HB 포함). 받은 메시지는 읽고 버린다(BYE와 RESTART는 처리). TCP 연결은 닫지 않는다.
- Buddy로의 CKPT 전송에서 오류(예외)가 나면 WARN만 남기고 계산을 계속한다(send_if_alive는 True로 간주). crash 직전에 계산 중이던 Task의 CKPT · RESULT가 새어 나가지 않도록, **Task를 꺼낼 때의 crash 세대 번호(crash_gen)**를 기억하고 step 전과 **전송 직전**에 비교한다(아래 의사코드).
- HB · Compute의 대기는 Condition Variable로(`state != RUNNING and not stopping`), BYE를 받으면 notify_all로 모두 깨워 종료한다. Worker 안에서 Lock을 겹쳐 잡을 때는 q_lock → state_lock → store_lock 순서.

```
# Worker 핵심 의사코드 (crash · rejoin · 종료 처리). HB Thread도 state != RUNNING and not stopping 동안 cv.wait()
def on_crash():
    with state_lock: state = CRASHED; crash_gen += 1
    with q_lock: local_q.clear(); requested = False; no_task = False
    with store_lock: store.clear()

def on_restart():                           # Peer Server가 Launcher의 RESTART를 받으면
    with state_lock: state = REJOINING
    send_coord(f"REJOIN {i}")               # RETRY_LATER면 2초 후, state가 아직 REJOINING일 때만 다시 (타이머 등으로)

def on_welcome():                           # Control Recv
    with state_lock: state = RUNNING; cv.notify_all()      # HB · Compute 깨우기
    send_launcher(f"REJOINED {i}")
    with q_lock: q_cv.notify_all()          # Queue가 비었으면 take_task()가 REQ

def on_assign(k, e):                        # Control Recv — REQ의 응답
    with q_lock: local_q.append((k, e, 1, 0)); requested = False; q_cv.notify_all()

def on_takeover(k, e, d):                   # Control Recv — 인계받은 Task
    with q_lock:
        with store_lock: ck = store.get(k)  # (epoch, step, partial) 또는 없음
        local_q.append((k, e, ck.step + 1, ck.partial) if ck else (k, e, 1, 0))
        q_cv.notify_all()
```

```
def on_no_task():                           # Control Recv
    with q_lock: no_task = True; requested = False

def on_bye():                               # Control Recv (CRASHED에서도 처리)
    with state_lock: stopping = True; cv.notify_all()      # HB Thread 깨움
    with q_lock: q_cv.notify_all()          # 기다리던 Compute 깨움
    log("TERMINATE", summary); close_all_sockets()

def send_if_alive(gen, target, msg):        # 확인과 전송 사이에 crash가 끼지 않게
    with state_lock:
        if state != RUNNING or crash_gen != gen: return False
    try: target.send(msg)
    except SendError: log("WARN", "send failed")   # Buddy(CKPT) 연결 오류만 WARN 후 계속 (2-3). Coordinator 연결 오류는 치명적 →
로그 후 종료
    return True

def take_task():                            # Task를 꺼내는 "순간"의 crash_gen을 함께 반환
    while True:
        wait_until_running_or_stop()
        with q_lock:
            with state_lock: running, gen = (state == RUNNING), crash_gen
            if stopping: exit_thread()
            if running and local_q: return local_q.popleft() + (gen,)
            need_req = running and not local_q and not requested and not no_task
            if need_req: requested = True
            else: q_cv.wait()               # ASSIGN · TAKEOVER · WELCOME · BYE가 notify
        if need_req: send_if_alive(gen, coord, f"REQ {i}")

def compute_loop():
    while True:
        (k, e, s0, partial, gen) = take_task()
        for s in s0..10:
            if crash_gen != gen: break      # 꺼낸 뒤 crash가 있었으면 버림
            partial += count_primes(k, s); sleep(step_delay)
            if s < 10 and not send_if_alive(gen, buddy, f"CKPT {k} {e} {s} {partial}"): break
        else: send_if_alive(gen, coord, f"RESULT {k} {e} {partial}")
```

③ Chaos Launcher (로컬, 1개)

- Worker 10개를 실행하고 각 Peer 포트에 제어 연결(첫 메시지 `LAUNCHER`, 거절되면 0.5초 후 재시도). READY 10개를 받은 시각을 t0로 하고 **seed 기반 난수**로 장애를 주입한다. Coordinator와는 통신하지 않는다.
- Worker마다 RUNNING / CRASHED / REJOINING / EXITED를 기록한다. REJOINED를 받으면 RUNNING, 제어 연결 EOF면 EXITED(예정 RESTART 취소). 10개 모두 EXITED면 Launcher.txt를 마감하고 종료한다.

규칙	값 (sec)	이유
victim 선택	그 순간 RUNNING인 Worker 중 무작위	이전에 crash했던 Worker도 다시 뽑힐 수 있다
첫 crash / 간격 I	t0 후 10 ~ 20 / 직전 CRASH 후 5 ~ 15	I가 D보다 짧으면 겹침 → 동시 2개 상황
동시 crash 상한	CRASHED · REJOINING이 2개면 REJOINED까지 대기(REJOINED 시각부터 간격 I를 새로 뽑음)	어느 순간이든 최대 2개
downtime D	CRASH 후 10 ~ 15 경과 시 RESTART	RESTART 전에 DEAD 판정(5~6)이 반드시 끝남
새 crash 중단	t0 후 80 이후 새 CRASH 없음	마지막 rejoin 때도 대개 나눠 줄 Task가 남음

④ 통신 프로토콜 (형식은 자유, 그대로 써도 됨)

TCP는 바이트 스트림이므로 Socket마다 수신 버퍼를 두고 줄바꿈(\n) 단위로 잘라 처리한다(한 번에 여러 메시지가 오거나 한 메시지가 나뉘어 와도 처리). 메시지 형식을 바꿨다면 Readme.txt에 설명한다.

방향	메시지	의미

W→C	`HELLO i ip port` · `RING_OK i` · `HB i` · `REQ i` · `RESULT k e v` · `REJOIN i`	등록(Peer 주소) · Buddy 연결 완료 · 1초 주기 생존 신호 · Task 요청 · 결과 · 재합류 요청
C→W	`RING j ip port` · `START` · `ASSIGN k 1` · `NO_TASK` · `TAKEOVER k e from=d` · `ACK k` · `REJECT_STALE k reason` · `WELCOME` · `RETRY_LATER` · `BYE`	Buddy 주소 · 계산 시작 · 배정 · 줄 Task 없음 · 인계(e≥2) · 반영 완료 · 거부(DONE/EPOCH/DEAD) · 재합류 승인 · 2초 후 재시도 · 종료
W→W	`PEER i` · `CKPT k e s partial`	Buddy 연결 직후 첫 메시지 · step s(1≤s≤9)까지의 누적 소수 개수
L↔W	`LAUNCHER` · `CRASH` · `RESTART` / `READY i` · `REJOINED i`	Launcher 제어 (Worker Peer 포트 경유)

⑤ 각 조가 자유롭게 정하는 사항 (Readme.txt에 설명)

- 소수 세기 구현 방법(권장: 1-2의 구간 체. 1-3의 검증 값과 일치하면 다른 방법도 가능), Worker의 Thread 구성, 로컬 Queue · Checkpoint Store 자료구조
- 인계받은 Task와 진행 중 Task의 처리 순서(권장: 진행 중 Task를 먼저 끝냄), REQ를 보내는 시점(ACK 뒤 또는 RESULT 직후. 3-②의 의사코드는 Queue가 비는 즉시 보내는 "RESULT 직후" 방식이며, ACK 뒤에 보내려면 ACK 수신 때 requested를 확인해 notify하도록 바꾼다)
- 메시지 형식, 합계 계산 방식(종료 시 Σ result[k] 권장 / Commit마다 누적), 정합성 확인 방법(6-2)
- 개발 편의를 위한 실행 인자(작은 step-delay, 수동 CRASH 명령 등). 단 제출용 정식 실행은 `--step-delay 100`과 3-③의 Launcher 규칙을 따른다.

4. 필수 구현 조건 및 감점 사유

※ 필수 구현 조건 (하나라도 미충족 시 과제 전체 0점)

- 모든 통신은 반드시 Socket(TCP)으로 구현되어야 함
- Coordinator는 별도의 물리적 원격 호스트(AWS, GCP 등)에서, Worker 10개와 Chaos Launcher는 같은 로컬 PC 한 대에서 실행하며, 최종 로그가 이 구성과 `--step-delay 100`에서 얻어진 것이어야 함
- Coordinator는 Dispatcher 6개의 고정 Thread Pool + Request Queue 구조여야 하며(요청/연결마다 Thread 생성 시 0점), Task Table은 Task별 Mutex로 보호해야 함
- Ring 구성(RING_OK 10개) 후에만 계산을 시작해야 함
- 가상 crash 동안 TCP 연결을 유지하고, DEAD는 Heartbeat 타임아웃으로만 판정해야 함(Launcher와 Coordinator 간 통신 금지)
- Chaos Launcher가 victim을 무작위로 정하고 임의의 순간 최대 2개까지 crash되는 실행에서, crash마다 DEAD → TAKEOVER(다음 ALIVE, CKPT가 있으면 다음 step부터) → REJOIN → 새 Task 수행(나눠 줄 Task가 남아 있는 경우)이 동작해야 함
- 종료 시 1,000 Task 모두 DONE, 모든 commit_count = 1(이중 Commit 0건), 합계 = 5,761,455

△ 주요 감점 사유

- 대기를 Condition Variable 대신 sleep 반복 확인(busy-waiting)으로 구현한 경우
- Lock 순서 위반, Lock을 잡은 채 send, REQ의 확인과 배정을 분리한 경우
- CKPT가 있는데도 항상 step 1부터 재계산, SUSPECT 단계 없음, 인계 대상이 다음 ALIVE 규칙을 따르지 않는 경우
- crash 중 HB · 메시지가 새어 나가거나, crash 때 상태를 지우지 않아 rejoin 후 옛 Task를 이어 계산하는 경우
- 로그에서 crash 주기(CRASH → DEAD → TAKEOVER → REJOIN → ASSIGN)를 추적할 수 없거나, Readme.txt의 필수 항목이 누락되었거나 실제 구현과 다른 경우

5. 시뮬레이션 시나리오

다음 시나리오에 따라 시스템을 동작시키고 Coordinator.txt, Worker1.txt ~ Worker10.txt, Launcher.txt에 모든 이벤트를 기록한다. 아래 시각은 START 기준(초)의 예시이다(seed = 42).

Step 1 - 기동과 Ring 구성: Coordinator 시작 → Launcher가 Worker 10개 실행 → HELLO 10개 → RING → P2P 연결(PEER i) → RING_OK 10개 → START(시각 0). 하나라도 빠지면 START는 나가지 않는다. Launcher는 READY 10개를 받은 뒤 스케줄을 시작한다.

Step 2 - 정상 계산: 각 Worker가 REQ → ASSIGN → step 1~10 계산(step마다 CKPT) → RESULT → ACK → 다음 REQ를 반복한다. 먼저 poll()된 REQ가 작은 번호를 받으며, 빨리 끝낸 Worker가 더 많이 가져간다.

Step 3 - 단일 crash와 인계: 예) 30.0초 W9 crash(마지막 HB 29.95, Task 298 step 6까지 CKPT) → 33.0 SUSPECT → 35.0 DEAD → W10에 `TAKEOVER 298 2 from=9` → W10이 step 7부터 계산 → 36.6 Commit. 41.0 RESTART → 41.02 WELCOME → 41.14 새 Task 402.

Step 4 - 이웃한 두 Worker의 동시 crash: 예) 15.0 W4 crash(152 step 6) → 20.0 W4 DEAD → 152를 W5에 인계(e2, W5는 방금 받은 160 계산 중이라 Queue에 둠) → 20.3 W5도 crash(160 step 3 CKPT는 W6에) → 25.0 W5 DEAD → W5가 owner인 160(e2)과 152(e3)를 모두 W6에 인계 → 160은 step 4부터, 152는 CKPT가 W5와 함께 사라져 step 1부터 → 25.93, 26.93 Commit.

Step 5 - 반복 crash와 Rejoin: 같은 Worker가 다시 뽑혀 crash할 수 있다(예: W4가 27초에 rejoin 후 42초에 다시 crash, 56초 rejoin_count = 2). Worker는 crash마다 모든 상태를 지우므로 몇 번째 crash인지 신경 쓸 필요가 없다.

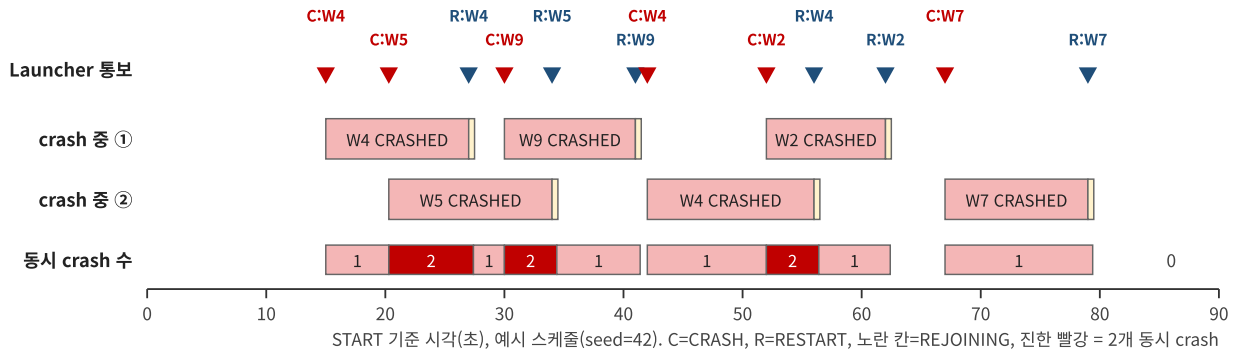


그림 15. 예시 crash 스케줄 (seed = 42) — W4 → W5 → W9 → W4 → W2 → W7. 진한 빨강 구간에서 2개가 동시에 crash 상태

Step 6 - 정식 실행: 원격 구성, `--step-delay 100`, Launcher 규칙대로 1,000 Task가 끝날 때까지 실행한다(약 100초 + crash 지연). 5초마다 POOL 로그를 남긴다.

Step 7 - 종료와 검증 (Graceful Termination): PENDING 소진 → REQ에 NO_TASK → 1,000번째 Commit → Main이 합계 · commit_count 검증(VERIFY, 이후 Failure Detector는 판정 중지) → 10개 모두에 BYE(CRASHED인 Worker도 BYE는 처리) → Worker 종료 → Launcher는 10개 EXITED 확인 후 종료 → Main이 STOP × 6으로 Dispatcher 종료 → 종료 플래그를 세우고 Listener(select 타임아웃 또는 Socket close로 깨움)와 Failure Detector(다음 주기에 플래그 확인)를 끝냄 → 모든 Thread join, TERMINATE. BYE 이후 Worker 연결의 EOF는 정상 종료로 보고 Socket을 닫는다(오류 · crash로 처리하지 않음).

6. 평가 지표와 정합성 확인

6-1. 측정 지표 (Coordinator.txt 최종 요약에 기록, CKPT 절약률은 Readme)

지표	정의 (측정 위치)
처리량 / 실행 시간	[Coord.] 1,000 ÷ (START ~ 마지막 COMMIT)
감지 지연 (crash마다)	[Coord.] 마지막 HB 수신 ~ DEAD 판정 (5~6초 예상)
복구 시간 (crash마다)	[Coord.] DEAD ~ 그 crash로 인계된 Task가 모두 Commit될 때까지 (인계할 Task가 없었으면 "—"로 기록)
재합류 지연 / rejoin 후 Task 수	[Coord.] REJOIN 수신 ~ rejoin 후 첫 ASSIGN / 재합류 후 그 Worker가 Commit한 Task 수
CKPT 절약률	crash별 Σ(재개 step − 1) ÷ (인계 Task 수 × 10). Worker 로그의 TAKEOVER 재개 step으로 계산해 Readme 표에 적고, 재개 로그가 없는(버려진) TAKEOVER는 0 (예: #1 = 6/10, #2 = 3/20)
이중 Commit	commit_count ≥ 2인 Task 수 (반드시 0)

6-2. 정합성 확인 (방법은 자유, Readme.txt에 설명)

① 1,000 Task DONE, commit_count 합 = 1,000 ② 합계 = 5,761,455 ③ Launcher의 CRASH 수 = Coordinator의 DEAD 수 = REJOIN 수(두 로그는 실제 시각으로 대조) ④ 어느 순간에도 DEAD 수 ≤ 2 (DEAD 로그에 그 순간의 DEAD 목록 dead_now를 기록

해 확인, 7장 예시) ⑤ crash마다 재합류 후 새 Task 수행(나눠 줄 Task가 남아 있는 경우) ⑥ Coordinator 로그의 COMMIT 수 = 1,000 (Worker의 ACK 수는 Commit 직후 crash하면 더 적을 수 있다). 모두 만족하면 PASS로 기록한다.

6-3. 결과 예시 (참고용 — 제출 시 실측값으로)

```
Elapsed : 101.2 s (START → last COMMIT)   Throughput : 9.88 task/s
Tasks   : done=1000 commits=1000 double_commit=0     Sum : 5761455 (expected 5761455) → PASS
Crashes : DEAD=6 REJOIN=6 TAKEOVER=7 REJECT_STALE=0 max_concurrent_dead=2
# worker lastHB  DEAD   detect  takeover(to)   recover  rejoin  1st_task
1 W4     14.95   20.00  5.05s   152→W5         6.9s*    27.02   271 (0.10s)
2 W5     19.95   25.00  5.05s   160,152→W6     1.9s     34.03   337 (0.09s)
3 W9     29.95   35.00  5.05s   298→W10        1.6s     41.02   402 (0.12s)
4 W4     41.96   47.00  5.04s   405→W5         1.3s     56.04   561 (0.08s)
5 W2     51.97   57.00  5.03s   523→W3         1.5s     62.03   618 (0.11s)
6 W7     66.95   72.00  5.05s   690→W8         1.4s     79.02   798 (0.10s)
(* #1의 152는 W5 crash로 한 번 더 인계되어 복구가 길어짐 — 5장 Step 4)
```

7. 노드별 Log 출력 형식

공통 형식: `[HH:MM:SS.mmm] NODE | EVENT | STATUS | message` (NODE = COORD, WORKERi, LAUNCHER / STATUS = SUCCESS, FAIL, INFO, WARN)
EVENT 타입: INIT, CONNECT, RING, START, ASSIGN, RESULT, COMMIT, CKPT, HB, SUSPECT, DEAD, TAKEOVER, CRASH, RESTART, REJOIN, WELCOME, REJECT_STALE, POOL, VERIFY, TERMINATE. message의 세부 내용은 각 조가 정하고 AllDefinedLogs.txt에 정리한다. crash마다 CRASH → DEAD → TAKEOVER → REJOIN → ASSIGN을 추적할 수 있어야 한다.

```
[14:30:00.000] COORD | START    | SUCCESS | ring ready (10/10 RING_OK). START sent to 10 workers
[14:30:20.000] COORD | DEAD     | INFO    | W4 no HB for 5.05s (lastHB=14.95). owned=[152] next_alive=W5 dead_now=[W4]
[14:30:20.001] COORD | TAKEOVER | SUCCESS | task=152 epoch 1->2 owner W4->W5
[14:30:25.000] COORD | DEAD     | INFO    | W5 no HB for 5.05s (lastHB=19.95). owned=[152,160] next_alive=W6 dead_now=[W4,W5]
[14:30:25.001] COORD | TAKEOVER | SUCCESS | task=152 epoch 2->3 owner W5->W6
[14:30:25.930] COORD | COMMIT   | SUCCESS | task=160 e2 by W6 value=5989 commit_count=1
[14:30:27.020] COORD | REJOIN   | SUCCESS | W4 DEAD->ALIVE rejoin_count=1. WELCOME
[14:31:41.200] COORD | VERIFY   | SUCCESS | done=1000 commits=1000 double=0 sum=5761455 PASS
# Worker6.txt
[14:30:25.010] WORKER6 | TAKEOVER | SUCCESS | task=160 e2 from W5. ckpt=(e1, step 3, 1794) -> resume step 4
# Launcher.txt
[14:30:20.300] LAUNCHER | CRASH   | INFO    | #2 victim=W5 (random from RUNNING=[1,2,3,5,…,10]) D=13.7s
```

8. 과제 제출물

G조이름HW3.zip (ex. G2HW3.zip; 조 이름 또는 조 번호 사용) // 압축 파일 오류 시 0점 처리

- 전체 소스 코드 (Coordinator, Worker, Chaos Launcher 포함)
- AllDefinedLogs.txt – 7장의 EVENT를 포함하여, 프로그램에서 정의한 모든 로그 메시지 명세 및 설명
- 결과 로그 파일: Coordinator.txt, Worker1.txt ~ Worker10.txt, Launcher.txt – 원격 구성에서 수행한 정식 실행 1회분이며, 모든 로그가 같은 실행에서 나온 것이어야 함
- download.txt – 5분 내외의 시연 동영상(G조이름HW3.mp4) 다운로드 가능한 링크 포함.
  - 링크 오류, 공유 권한 없음, 재생 오류는 해당 조 과실로 감점
- Readme.txt – 아래 항목 포함:
  - 조원 이름 · 학번 · 역할 명시
  - 배포 정보: 원격 호스트(서비스명, 인스턴스 유형, OS), Coordinator/Worker/Launcher 실행 명령, 사용한 시간대, seed. 공인 IP는 가려도 됨
  - 프로그램 구성요소 설명 및 메시지 형식, 소스코드 컴파일 및 실행 방법 (컴파일 · 실행 불가 시 과제 전체 0점)
  - 장애 감지(HB · FD) · Checkpoint · Takeover · Epoch · Rejoin 설계, Worker의 crash 처리 방법 설명
  - 인계받은 Task와 진행 중 Task의 처리 방법 설명
  - crash별 지표 표(6-1)와 6-2 정합성 확인 방법 · 결과
  - 프로그램 구현에 있어서 추가로 언급할 내용 및 한계, 추가 구현 사항 등 자유롭게 기술